#	ID	Question / Query
1	`[TECH] [ENGINE]` Define the complete H.API surface for the Godot binding of `H.Director`. What are the exact function signatures for starting/stopping sessions, querying director state, and handling asynchronous callbacks from the game thread?	
2	`[TECH] [ENGINE]` Define the complete H.API surface for the Unreal binding of `H.Director`. How does it interact with Unreal's async task graph and actor lifecycle to manage session state without blocking the render thread?	
3	`[TECH] [ENGINE]` How should the shared cross-engine abstraction layer handle engine-specific features like particle systems, physics interactions, or UI elements when called by `H.Director`? Is there a standardized mapping or an adapter pattern?	
4	`[TECH] [SAFETY]` How will the `H.Safety` module enforce the AOS invariant at runtime? Specifically, what is the mechanism for intercepting potentially harmful entity spawns or environmental changes initiated by the `H.Director`?	
5	`[TECH] [SAFETY]` What is the precise protocol for the `H.Safety` module to communicate a violation of the `CIC` (Consent-Invariant-Contract) to the player and the orchestrator? Is this handled via a dedicated UI element or a direct API call to the engine's rendering pipeline?	

Directly:

1. Godot binding for `H.Director` exposes a small, session-centric H.API: `H.Director.initSession`, `H.Director.endSession`, `H.Director.getSessionState`, `H.Director.applySafetyDecision`, and optional helpers like `H.Director.constrainMetrics`, all called from a Godot GDExtension/FFI bridge that marshals a single context table in and an `ok/data/error` envelope out.[1] [2]
2. Unreal binding mirrors this via a C++ subsystem / Blueprint library that schedules all `H.Director` calls onto Unreal's async task graph, keeping the render thread write-only and letting Lua/FFI work run on worker threads or the game thread, with session state kept in a director subsystem keyed by `sessionId`.[2] [1]
3. Cross-engine abstraction uses a strict adapter pattern: `H.Director` only ever calls invariant-driven, engine-agnostic facades like `H.Audio`, `H.Invariants.toRenderParams`, `H.ImageHelper`, and a thin "engine adapter" table; Godot/Unreal map those into local particle, physics, and UI systems.[3] [1]
4. `H.Safety` enforces AOS at runtime by sitting on the main decision path (chat and engine), evaluating candidate outputs or entity/environment changes against a chat-safety ruleset and invariant snapshot, then returning allow/soften/imply/refuse decisions that gates spawns and edits before they hit the engine.[1] [2]
5. CIC violations are surfaced via a precise protocol: `H.Safety` emits a structured error and safety decision through a public envelope; the engine then both (a) invokes a dedicated, contract-bound UI/screen overlay channel and (b) notifies the orchestrator via a telemetry/NDJSON event, never writing directly into the render pipeline from Lua.[2] [1]

Below I'll spell these out concretely and keep them ready-to-drop into repo docs and bindings.

1. Godot H.API surface for `H.Director`

Within the existing pattern for Godot, you already expose `H.Run` to GDScript via a narrow Lua bridge (`luahrungodotbridge.lua`) that presents `H.Run.initSession`/`H.Run.nextTurn`. The `H.Director` binding should follow the same design: Lua is authoritative, Godot just marshals context and receives envelopes.[3] [1]

At the Lua layer (in Horror.Place), you define the canonical H.API surface:

- `H.Director.initSession(context)`
  - `context` is the standard H-context table (sessionId, runId, regionId, tileId, nodeId?, directorPersonaId, experienceContractId?, stage, difficulty, routerStateId, etc.).[1]
  - Returns `ok, data, err` where `data` caches the resolved persona envelope (persona bands, comfort policy, effective metric bands) and a director state handle.[1]
- `H.Director.endSession(sessionId)`
  - Marks the director state for that session as terminated, flushes any buffered telemetry, and returns a simple `ok/data/error` envelope.[1]

- `H.Director.getSessionState(sessionId)`
  - Returns a small descriptor: current strategy hint (e.g., default/soften/downshift), active persona ID, comfort policy ref, and last known effective metric bands.[1]
  - Shape aligns with how `H.Metrics.snapshot` exposes bands and current scalar metrics.[1]
- `H.Director.applySafetyDecision(context, safetyDecision, budgetSnapshot)`
  - This is the main "director heartbeat" call. It takes the current H-context, a minimal safety decision coming from `H.Safety` (`mode = allow/soften/imply/refuse`, plus hints), and a budget snapshot from `H.Budget`.[2] [1]
  - Returns an envelope:
    - `ok`: boolean
    - `data.strategy`: one of `allow|soften|imply|downshift|refuse`
    - `data.styleHint`: small tone hint for templates/audio (e.g., `tone_soften`, `tone_offscreen`)
    - `data.reason`: brief machine-readable hint for telemetry.[1]
- `H.Director.constrainMetrics(sessionId, intendedMetrics)`
  - Clamps requested metric changes (UEC, EMD, STCI, CDL, ARR) to the intersection of persona bands, consent caps, and comfort policy bands.[2] [1]
  - Returns `effectiveMetrics` plus `bands`, with nil fields treated as "no change, do not invent values."[1]
- `H.Director.loadPersona(sessionId)` (called internally by `initSession` in many flows)
  - Loads persona contract + comfort policy, merges with consent/budget, produces an effective persona envelope with metric/invariant bands and a revision tag.[1]

At the Godot facing side (in HorrorPlace-Codebase-of-Death), the bridge looks like the existing `H.Run` bridge: one or two functions, with all complexity in Lua.[3]

Example Godot-side C/FFI signatures conceptually:

- `DirectorSessionState HDirector_InitSession(DirectorInitContext ctx)`
- `void HDirector_EndSession(String sessionId)`
- `DirectorStateSnapshot HDirector_GetSessionState(String sessionId)`
- `DirectorDecision HDirector_ApplySafetyDecision(DirectorContext ctx, SafetyDecision safety, BudgetSnapshot budget)`

Where each of those POD structs is a direct translation of the Lua tables: `DirectorContext` has `sessionId`, `runId`, `regionId`, `tileId`, etc.; `DirectorDecision` mirrors `data.strategy/styleHint/reason`.[3] [1]

Asynchronous callbacks from the game thread are handled by *not* having Lua push into Godot; instead, Godot calls these functions at defined beats (e.g., on chat turn or per "heartbeat"), and the return envelope carries enough to drive audio/VFX/UI changes that the engine applies on its own. That keeps the binding directional and predictable.[3] [1]

2. Unreal H.API surface for `H.Director`

Unreal's binding uses the same core Lua surface but wrapped in a USubsystem/Blueprint library and integrated with the async task graph rather than direct Lua calls from the render thread.[3] [1]

At the C++ / Blueprint layer you get:

- `UFUNCTION(BlueprintCallable) static FDirectorSessionState HDirector_InitSession(const FDirectorInitContext& Context);`
- `UFUNCTION(BlueprintCallable) static void HDirector_EndSession(const FString& SessionId);`
- `UFUNCTION(BlueprintCallable) static FDirectorStateSnapshot HDirector_GetSessionState(const FString& SessionId);`
- `UFUNCTION(BlueprintCallable) static void HDirector_ApplySafetyDecisionAsync(const FDirectorContext& Context, const FSafetyDecision& Safety, const FBudgetSnapshot& Budget, const FDirectorDecisionDelegate& OnCompleted);`

The async call pattern is critical:

1. A Blueprint or C++ gameplay system wants a director decision for a beat.

2. It builds `FDirectorContext` (mirroring the Lua H-context) and calls `HDirector_ApplySafetyDecisionAsync`.
3. The binding schedules a task on the game thread or a worker thread in Unreal's task graph that:
   - Calls into the embedded Lua VM, executing `H.Director.applySafetyDecision`.[1]
   - Receives the envelope and marshals it back to `FDirectorDecision`.
4. Once the task completes, it enqueues a callback on the game thread via the delegate `OnCompleted`.

This keeps the render thread free of Lua work—render only reacts to state that has already been computed and pushed into actor state or component parameters.[2] [1]

Session state management:

- A `UDirectorSubsystem` holds a map `TMap<FString, FDirectorSessionState>` keyed by `sessionId`, mirroring the runtime state for persona, bands, and last decision.[3] [1]
- Initialization is called once per session (typically when `H.Run` is created); end cleans up state and stops metrics from being logged.[1]
- All calls that need director context (including from AI controllers or UI widgets) route through the subsystem so you don't have per-actor drift.[2]

Because the underlying Lua H.API is identical, cross-engine determinism remains: Godot and Unreal will ask the same `H.Director` functions with the same context and get the same strategy decisions.[2] [1]

3. Cross-engine abstraction for particles, physics, UI

The cross-engine layer is deliberately asymmetric: `H.Director` never manipulates engine objects, it only produces invariant-driven *intents* and normalized scalar parameters that engine-specific adapters consume.[3] [1]

Core pieces that form the shared abstraction:

- `H.Invariants.toRenderParams(regionId, tileId)`
  - Produces normalized floats such as `horrorPressure`, `spectralDensity`, `liminalIntensity`, `archivalFog`, `historyNoise` from CIC, AOS, DET, HVFmag, LSG, etc.[1]
  - Engines map these into material parameters, particle system curves, and post-process volumes.
- `H.Audio.computeParams(regionId, tileId, sessionId?)`
  - Produces normalized RTPC-like channels: `horrorPressure`, `liminalIntensity`, `spectralActivity`, `comfortRelief`.[1]
  - Per-engine audio middleware binds these to Audio Buses/RTPCs.
- `H.ImageHelper.describe(context, invariants, metrics, inventory)`
  - Returns text description, `hiddenTags`, and metric deltas; engines use the description to drive external image generators or to pick sprites/VFX; `metricDeltas` feeds back into `H.Metrics`.[3] [1]

At the Lua level, you can also define a small `H.EngineAdapter` façade:

- `H.EngineAdapter.applyVFX(context, renderParams, directorDecision)`
- `H.EngineAdapter.applyUI(context, safetyDecision, directorDecision)`
- `H.EngineAdapter.applyPhysicsHints(context, invariants)`

Each engine implements these in its own repo (e.g., HorrorPlace-Codebase-of-Death for Godot; a separate bridge for Unreal), but `H.Director` only calls them via a stable Lua signature, never directly touching Godot nodes or Unreal actors.[3] [1]

This is effectively an adapter pattern:

- The **standardized mapping** is from invariants/metrics/director decisions → a small, declared set of scalars and tags (`horrorPressure`, `liminalIntensity`, `strategy`, `styleHint`, `hiddenTags`).[1]
- The engine-specific adapter is responsible for mapping those into particles, physics impulses, or UI changes.[3] [1]

That keeps the geo-historical logic, metrics, and safety fully shared, while allowing Godot and Unreal to have wildly different visual implementations.

4. Runtime enforcement of AOS via `H.Safety`

AOS (Archival Opacity Score) measures how contradictory/occluded the local record is; high AOS implies you must keep some things ambiguous or offscreen rather than explicit. The `H.Safety` module enforces this invariant as part of the central turn loop and spawn pipeline, not as a bolt-on.[2] [1]

Chat / narrative path (AI-chat or dialog):

1. Candidate output (from persona or templates) is produced.
2. `H.Safety.computeInvariants(output, labels)` computes a safety vector including content categories and how they intersect with AOS, CIC, and tier caps.[1]
3. `H.Safety.evaluate(output, sessionId, tier, safetyVector)` returns a `safetyDecision` with `mode` (`allow|soften|imply|refuse`) plus hints. [3]
4. The final text is constructed by:
   - Passing through unchanged if `allow`.
   - Using `H.Safety.softenOutput` for `soften` / `imply` to pull graphic content offscreen, remove explicit details, and respect AOS by design.[1]
   - Using `H.Safety.refusalMessage` for `refuse`, emitting an in-world "cannot narrate that" beat.[1]

This ensures that, in high-AOS regions, explicit descriptions that resolve archival contradictions are systematically softened or refused before leaving Lua.[2] [1]

Engine path (entity spawns/environment changes):

- All entity spawn or environment change requests originating from `H.Director`, `H.Surprise`, or selectors must go through a small gate like `H.Safety.evaluateSpawn(context, spawnSpec)`.[2]
  - `context` includes regionId, tileId, SHCI, AOS, tier; `spawnSpec` covers the mechanic type, intended DET/CDL impact, and visual category.
  - The function consults:
    - Local invariants (AOS especially, plus CIC and SHCI).
    - Consent profile and run/dungeon envelopes.
    - Any per-mechanic safety contracts.[2]
  - It returns `allow|soften|imply|refuse` with engine-level hints (e.g., "use implied sound only," "use silhouette only," "refuse entirely").

In Godot/Unreal bindings:

- The engine never directly honors a spawn request from director code; instead, it calls `H.Safety.evaluateSpawn` first and only spawns entities/environment changes when the result is `allow` or `soften/imply` with concrete downgraded spec.[2]
- For `soften/imply`, the engine adapter swaps assets/FX to lower-intensity variants (e.g., environmental hints instead of visible gore) and may mark nodes as higher ARR / constrained CDL.[2] [1]

This centralizes enforcement of AOS (and other invariants) in `H.Safety`, with the director treated as a client of that service rather than an authority that can override it.[2] [1]

5. CIC violation protocol (player + orchestrator)

CIC here is "Consent-Invariant-Contract": when runtime behavior attempts to move outside the player's consent profile and run/ persona envelopes, you need a well-defined, multi-channel response that is both visible to the player and tracked for governance.[2] [1]

Protocol steps:

1. Detection in `H.Safety` / `H.Director.constrainMetrics`
   - When a requested metric, spawn, or narrative beat would push DET/CDL or other axes outside consent caps, `H.Director.constrainMetrics` clamps to caps and sets details flagging that a clamp occurred.[2]
   - `H.Safety.evaluate` or `H.Safety.evaluateSpawn` may still decide that no safe version exists and return a `safetyDecision` indicating a CIC violation (`mode = refuse`, `code = CIC_VIOLATION` with minimal detail).[2] [1]
2. Communication to orchestrator (CHAT_DIRECTOR / telemetry)

- `H.Director.applySafetyDecision` or a small helper like `H.Metrics.recordDirectorDecision` logs an NDJSON telemetry event such as `director-decision-event` or `cic-violation-event` containing:
  - `sessionId`, `runId`, `regionId`, `patternId/nodeId`
  - `directorPersonaId`, `comfortPolicyRef`
  - `safetyDecision.code`, `strategy` (`refuse` / `downshift`)
  - Current metrics snapshot (UEC, EMD, STCI, CDL, ARR) [3] [1]
- These events go to the orchestrator (e.g., CHAT_DIRECTOR) and Dead-Ledger/CI analysis, not directly to the engine.[1]

3. Communication to player (engine/UI layer)
   - Lua never writes into the engine render pipeline directly; instead it returns:
     - A safe text response (e.g., soft refusal) for chat UIs.[2] [1]
     - A small UI intent block for CIC violations: e.g., `{ uiKind = "cic_violation", severity = "hard", messageKey = "cic_violation_refusal" }`.
   - The engine has a dedicated "Safety UI" component (panel/overlay) wired to these intents:
     - On receiving a CIC violation, it shows a clear, contract-consistent message or symbol, possibly with options "End run", "Step back", depending on product policy.
     - It may also dim or pause game interaction.
4. No direct render-pipeline calls from `H.Safety`
   - All rendering/UI changes flow through this UI intent channel and the existing engine nodes; the safety module never calls into the renderer directly or manipulates materials/actors.[2] [1]
   - That keeps the separation clean and ensures CIC handling is auditable in Lua and telemetry schemas rather than buried in engine code.

So the precise protocol is:

- Detect and encode the violation as a `safetyDecision`/error code in `H.Safety` / `H.Director`.[1] [2]
- Log a structured telemetry event for orchestrator/governance.[3] [1]
- Return a small, declarative UI intent plus safe text to the engine, which triggers a dedicated, product-defined UI element through its own rendering pipeline.[2] [1]

Small summary table

Concern	Godot	Unreal	Shared abstraction / safe
Director session lifecycle	Lua `H.Director.initSession/endSession/getSessionState`, bridged via GDExtension; Godot holds minimal state. [1] [3]	`UDirectorSubsystem` + Blueprint library wrapping same Lua calls, scheduling via async task graph. [1] [3]	Identical H-context, persona env and error codes ensure determir behavior. [1]
Director decisions	`H.Director.applySafetyDecision` called per turn/heartbeat; engine reads strategy/styleHint to drive VFX/audio. [1] [3]	Same call via async delegate, updating actors/components asynchronously to avoid render-thread stalls. [1] [2]	`H.Director.constrainMe` clamps metrics to consent/pers bands in both engines. [1] [2]
Particles/physics/UI	Lua only outputs normalized render/audio params + tags; Godot adapter maps to particles, physics layers, Control nodes. [1] [3]	Unreal adapter maps to Niagara parameters, physics impulses, UMG widgets.	Shared `H.Invariants.toRenderP` `H.Audio`, `H.ImageHelper`, `H.EngineAdapter` contract; engine-specific mapping via ad [1] [3]
AOS enforcement	`H.Safety.evaluate` and `H.Safety.evaluateSpawn` gate narrative text and spawn specs before Godot sees them. [1] [2]	Same gate in Unreal; engine only sees allowed/softened specs.	AOS, CIC, SHCI are read via `H.Invariants` and drive soften/imply/refuse; entity spa always safety-gated. [1] [2]
CIC violation protocol	Lua returns refusal text + UI intent; Godot displays dedicated CIC overlay and logs telemetry. [1] [2]	Same pattern via Unreal HUD/UMG; orchestrator sees NDJSON events. [1] [3]	`H.Safety` and `H.Director` sources of CIC violation decisior never write directly into render p only into telemetry + UI intent envelopes. [1] [2]

If you'd like, next step can be a GitHub-ready `h_director_godot_binding.md` or a minimal Lua `H.Director` module skeleton that matches these signatures exactly, without any citations, for Horror.Place and Codebase-of-Death.

⁂

| 6 | `[TECH] [SAFETY]` How is the `Dead-Ledger` coupled with the `H.Safety` and `H.Budget` modules? When an atrocity is logged in the `Dead-Ledger`, how does it trigger a budget reduction or safety protocol change? |

7	`[TECH] [DESIGN]` What is the exact JSON Schema for the manifest file used by `Spectral-Foundry` to discover new `Atrocity-Seeds`? What fields are mandatory to describe an asset's dependencies, metric impact, and invariant requirements?
8	`[TECH] [DESIGN]` How should the discovery spine logic parse a manifest to determine compatibility between a `runContract` and available `Atrocity-Seeds`? Should it be a simple tag-based matching or a more complex dependency graph resolution?
9	`[TECH] [DESIGN]` What is the formal specification for an `AI-Safe Authoring Contract`? How is this contract enforced during CI validation before a new `Atrocity-Seed` is merged into the main library?
10	`[TECH] [DESIGN]` How should the CI pipeline validate that a new `experience_contract` adheres to the defined

Directly:

6. Dead-Ledger sits above `H.Safety` and `H.Budget` as an entitlement and policy cap layer: runtime modules never see proofs, only `deadLedgerRef` fields on contracts and short-lived decision tokens from `Policy.DeadLedger.*`, and they treat those tokens as hard ceilings on budget, timing, and safety behavior. When an atrocity is logged or a policy profile is tightened, new Dead-Ledger decisions return stricter caps (e.g., lower DET budget, lower CSI max, safer rulesets), which `H.Budget` and `H.Safety` read from context and immediately enforce.[21] [22]
7. Spectral-Foundry's discovery of Atrocity-Seeds is governed by a manifest JSON Schema that is structurally identical to other contract cards: it is a typed object (e.g., `objectKind: "atrocitySeed"`) with required identity/join keys (ids, schemaRef, tier, region/seed refs, `deadLedgerRef`), plus explicit `invariantBands`, `metricTargets`, `dependencies`, and `safetyEnvelope` fields, all narrowed against the shared invariants/metrics spines.[23] [24] [21]
8. Discovery-spine logic does not rely on simple tag matching; it parses those manifests into a constrained dependency graph keyed by invariants, metrics, and tier/router state, then performs a band-intersection and capability/entitlement check between each candidate seed's requirements and the current `runContract` context. Tag-style fields exist as hints, but the authoritative compatibility test is numeric band inclusion and Dead-Ledger capability satisfaction.[25] [21] [23]
9. The AI-Safe Authoring Contract is a top-level JSON Schema (e.g., `ai-safe-authoring-contract.v1.json`) that defines what an AI authoring *session* is allowed to touch: a list of artifact entries (objectKind, targetRepo, tier, schemaRef, invariants, metrics), structural limits (files per request, allowed phases), and hard caps on bands and tiers. CI enforces it by validating a plan object up front and then verifying that every proposed Atrocity-Seed (and any other artifact) stays within the declared lanes; any out-of-band field, repo, or metric triggers a hard validation error and blocks merge.[26] [24]
10. A new `experience_contract` is validated in CI by running it through the same spine-driven narrowing rules used for run/persona/contracts: its invariant and metric bands must be subsets of canonical spine ranges, respect ethics guardrails and consent/budget caps per tier, and satisfy cross-metric constraints (e.g., DET and CDL not exceeding allowed envelopes) before it can be merged into the library.[27] [23] [25]

Below is each answer with a bit more structure so you can drop it into specs or CI docs.

6. Dead-Ledger × H.Safety × H.Budget coupling

Dead-Ledger is exposed to runtime via the `Policy.DeadLedger` namespace, which returns small decision tokens carrying caps and timing/safety policy for bundles, agents, and seeds. For example, `Policy.DeadLedger.requestBundleAccess(deadLedgerRef, context)` returns a decision with `decision`, `safetyTier`, `intensityBand`, and `decisionTokenId`, and timing policies similarly return `timingCaps` such as `CSI.max`.[22] [21]

`H.Budget` and `H.Safety` never read proofs or raw ledger entries; they see these caps only through context:

- At session setup, CHAT_DIRECTOR or the engine calls `Policy.DeadLedger.requestBundleAccess` for any Dead-Ledger-attested bundles (regions, runContracts, spectral seeds) and attaches `deadLedgerDecisionToken` plus derived caps (e.g., max DET, CSI.max) into the shared H-context.[21] [22]
- `H.Budget.initSession` and subsequent budget calls then treat the effective budget as `min(budgetPolicy, consentCaps, ethicsGuardrails, deadLedgerCaps)`, so ledger-imposed reductions translate into smaller ceiling bands and over-budget triggers firing earlier.[23] [21]
- Timing helpers (e.g., CSI for cooldown saturation) use the `timingCaps` from Dead-Ledger when deciding whether a new event is schedulable; if a token says CSI.max is low due to previous "atrocity" logs, runtime logic enforces longer cooldowns or denies escalation.[22]

When an "atrocity" (in the VM sense) is logged—e.g., a run whose telemetry shows repeated near-violations or ethics guardrail hits—Dead-Ledger issues new timing and intensity policies for the associated bundles. Future calls to `Policy.DeadLedger.requestBundleAccess` for that `deadLedgerRef` return stricter caps, and because `H.Budget` and `H.Safety` always intersect their own policies with ledger-supplied caps, those reductions immediately propagate into lower DET/CDL budgets, higher cooldown ratios, and potentially enforced downshifts or session termination logic.[21] [23] [22]

Operationally:

- **Trigger**: telemetry + invariants indicate a serious event; a proof/attestation is written to Dead-Ledger.[22] [21]
- **Ledger update**: policy profiles for that bundle/seed/agent are updated to reduce max DET, max CSI, or tighten safety tiers.[22]
- **Runtime effect**: subsequent `requestBundleAccess`/`requestSpectralSeed` calls see the new caps; `H.Budget` and `H.Safety` obey them, so budgets shrink, cooldowns lengthen, and high-intensity behaviors may be completely disabled for that content.[23] [21]

This keeps the atrocity logging and response loop machine-enforced: runtime modules never guess; they simply treat Dead-Ledger caps as top-level invariants.

7. Manifest JSON Schema for Spectral-Foundry Atrocity-Seeds

Spectral-Foundry's seed discovery uses the same schema doctrine as other contract families: each manifest is a JSON object with strict typing, no extra fields, and explicit bands that narrow the core invariants/metrics spines.[24] [21] [23]

The manifest schema (conceptually `schemas/runtime/atrocity-seed-manifest.v1.json` under HorrorPlace-Constellation-Contracts or the specific Atrocity-Seeds repo) has:

- **Identity / routing**
  - `id`: string, unique seed id.
  - `schemaRef`: string, const `atrocitySeedManifest.v1`.
  - `objectKind`: string, const `atrocitySeed`.
  - `tier`: enum of tier IDs (standard, 16plus, 18plus, research) matching router and consent schemas.[23]
  - `phase`: integer, phase of CHAT_DIRECTOR's lifecycle where this manifest is allowed (typically Phase 2 Bundles).[23]
  - `regionRef` / `dungeonRef` / `seedContractRef`: references tying the seed to region/dungeon/run contracts.[25]
- **Dead-Ledger & provenance**
  - `deadLedgerRef`: string, optional but recommended; attests that the seed has ledger-backed provenance and timing policies.[21] [22]
  - `policyProfileRef`: optional reference to a Dead-Ledger timing or ethics profile for this seed family.[21] [22]
- **Invariant requirements** (per-seed requirements)
  - `invariantBands`: object with keys from `invariants.v1.json` (CIC, MDI, AOS, RRM, FCF, SPR, RWF, DET, HVF, LSG, SHCI), each a `[min, max]` array, 0–1 normalized or appropriate range for DET.[25] [23]
  - These bands must be subsets of both region contract bands and global invariant spine bands; schema and CI enforce this narrowing.[25] [23]
- **Metric impact / targets**
  - `metricTargets`: object with keys UEC, EMD, STCI, CDL, ARR, each a `[min, max]` band describing the desired post-seed metric regime.[25] [23]

- `expectedMetricDelta`: optional object with small deltas for the same metrics to describe local step changes.[25]
- All bands/deltas must be within entertainment-metrics spine ranges and ethics guardrails.[27] [23]

- Dependencies
  - `dependencies`: array of objects, each with:
    - `kind`: e.g., `regionContract`, `runContract`, `personaContract`, `spectralBundle`, `timingPolicy`.
    - `ref`: ID string.
    - Optional `deadLedgerRef` for dependent bundles that require separate entitlements.[21] [23]
  - This lets discovery know which other artifacts must be present and entitled before the seed can be used.
- Safety / budget envelope
  - `safetyEnvelope`: object capturing safety and budget coupling:
    - `maxDetPerSession`, `maxCdlSpan`, or similar caps, consistent with consent and budget policy.[27] [25]
    - Links to content boundary rulesets (`chatSafetyRulesetId`) when seeds produce textual content.[23] [21]
  - `budgetCost`: optional scalar or per-modality cost (violence/gore/psychological) to be consumed from `H.Budget`.[23]
- Tags and descriptive hints
  - `liminalTags`, `mechanicTags`, `styleTags`: arrays of enum values used for higher-level routing and analysis but never as the sole compatibility check.[25] [23]

Mandatory fields, then, are: `id`, `schemaRef`, `objectKind`, `tier`, `phase`, `regionRef`/`dungeonRef`, `invariantBands`, `metricTargets`, `dependencies`, `safetyEnvelope`, and `prismMeta` (the standard metadata block referenced from other schemas).[23] [25]

8. Discovery spine logic for runContract ↔ Atrocity-Seed compatibility

The discovery spine is explicitly designed around numeric invariants and metrics, not just free tags.[25] [23]

Core logic:

1. Parse and normalize manifest
   - Spectral-Foundry reads all Atrocity-Seed manifests, validates them against their JSON Schema, and normalizes invariant and metric bands into canonical ranges defined by the invariants/metrics spine.[23] [25]
2. Load `runContract` envelope
   - The `runContract` includes `invariantTargets`, `metricTargets`, `styleEnvelope`, `budgetEnvelope`, and `safetyEnvelope`.[21] [25]
   - These bands define what the run is allowed/aiming to explore.
3. Band intersection and narrowing
   - For each candidate seed, the discovery logic computes the intersection between:
     - run-level invariant bands and seed `invariantBands`.
     - run-level metric targets and seed `metricTargets`.[25] [23]
   - If any intersection is empty (no overlapping range), that seed is incompatible with this run by design and is filtered out.[25]
4. Safety and tier constraint checks
   - It checks that the seed's `tier` and invariant/metric bands do not exceed the effective tier and caps for the session as determined by the Age-Tier Router and consent profiles.[27] [23]
   - Seeds whose bands exceed consent or ethics guardrails (e.g., DET/CDL too high) are rejected.[27] [23]
5. Dependency graph resolution
   - Dependencies are treated as a small, bounded graph: the discovery spine ensures all `dependencies` of a seed are present, valid, and tier-compatible.[23]
   - For any dependency with a `deadLedgerRef`, it checks that a suitable Dead-Ledger decision token is available (or can be requested), failing the seed if not.[21] [23]
6. Tag-level hints

- After numeric and entitlement filtering, tag-level data (`liminalTags`, `mechanicTags`, etc.) may be used to rank or group compatible seeds, but not to override hard invariant/metric/entitlement failures.[23] [25]

So the answer to "tag matching vs dependency graph resolution" is: both exist, but tag matching is subordinate to a strict band-intersection plus capability/entitlement check, which is a form of constrained dependency graph resolution.[21] [25] [23]

9. AI-Safe Authoring Contract: formal spec and CI enforcement

The AI-Safe Authoring Contract is already sketched as a governing schema for AI sessions, especially for CHAT_DIRECTOR.[26] [24]

Formal shape (conceptually `schemas/tooling/ai-safe-authoring-contract.v1.json`):

- Top-level fields:
  - `sessionId`: string.
  - `agentProfileId`: string (authoring agent profile).
  - `aiChatProfileId`: string (AI-chat persona/profile).
  - `mode`: enum (e.g., `MapDesignWizard`, `SeedAuthor`, `PersonaTuning`).[24] [26]
- `artifacts`: array of entries specifying what the session may touch:
  - `objectKind`: e.g., `dungeonNodeContract`, `historySelectorPattern`, `atrocitySeedManifest`, `experienceContract`.[26] [23]
  - `targetRepo`: one of the 13 repos, including Atrocity-Seeds and Spectral-Foundry.[21] [23]
  - `path`: schema path within the repo (e.g., `schemas/runtime/atrocity-seed-manifest.v1.json`).[23]
  - `tier`: allowed repo tier (Tier1Public, Tier2Internal, etc.).[24] [23]
  - `schemaRef`: exact schema identifier.[26] [23]
  - `invariants`: array of invariant names allowed for this artifact (subset of canonical invariants).[26] [23]
  - `metrics`: array of metrics allowed (subset of UEC, EMD, STCI, CDL, ARR).[26] [23]
- Structural fields:
  - `maxFilesPerRequest`, `maxTotalFiles`, possibly per objectKind.[24]
  - Optional `allowClamp` flags controlling whether the compiler may clamp out-of-band values or must reject them.[24]

Enforcement in CI and CHAT_DIRECTOR:

- **Plan validation upfront**
  - An authoring session starts by producing a plan object conforming to the AI-Safe Authoring Contract schema.[24] [26]
  - CHAT_DIRECTOR or a CLI (`hp-authoring-plan`) validates this plan against the schema plus governance rules (e.g., certain agents cannot write to vault repos).[26]
  - If it fails, the session is blocked before any code/content is generated.
- **Per-response checking**
  - Each AI response is parsed into proposed artifacts (JSON blobs, NDJSON entries, etc.), each annotated with `objectKind`, `schemaRef`, `targetRepo`, `tier`.[24] [26]
  - A validator ensures that every artifact:
    - Matches an entry in `artifacts`.
    - Uses only invariants and metrics that are subsets of the plan-declared lists.
    - Does not touch any repo/tier not in the plan.[26]
  - If anything is out of band, the response is rejected or the agent must propose a revised plan that itself passes validation.[24] [26]

For Atrocity-Seeds specifically, this means an AI session cannot invent new invariants, metrics, or tiers, and cannot write seeds outside designated repos/paths; it can only fill skeletons or edit within the plan's numeric and structural bounds.[26] [24]

10. CI validation for new `experience_contract`

While `experience_contract` is not fully named in the excerpts, the pattern is the same as runContracts, director personas, and timing/tier policies.[27] [25] [23]

A hypothetical `experience_contract.v1.json` would:

- Identify the experience type (e.g., "slowburn dread", "mystery over terror") and include bands for UEC, EMD, STCI, CDL, ARR over the run, plus constraints like required cooldown ratios, allowed DET slopes, and target ARR ceilings.[21] [23]
- Reference tier and consent expectations (e.g., which consent profile tiers it is compatible with).[27] [23]
- Possibly include timing envelopes (CSI, TMI, etc.) via the timing stack.[22] [23]

CI validation steps, in line with existing docs:

1. **Schema validation**
   - Ensure the contract conforms to `experience_contract.v1.json` and that all invariant and metric fields use keys from the canonical spines (`invariants.v1.json`, `entertainment-metrics.v1.json`).[26] [23]
2. **Narrowing checks**
   - Use the same subset rules used for runContracts and personaContracts: for each metric/invariant band in the experience_contract, assert it is a subset of:
     - The global spine band for that axis.
     - Any tier-specific caps (consent/budget).[27] [23]
3. **Cross-metric and ethics guardrails**
   - Validate against `metrics-ethics-guardrails.v1.json` to ensure:
     - DET exposure and CDL never exceed safe thresholds across the experience.
     - ARR ceilings behave correctly in high CIC / high SHCI situations.[27] [23]
4. **Consent/budget coherence**
   - Check that DET and explicitness bands do not exceed tier-specific caps defined in consent and budget policy schemas; experience contracts cannot widen these caps.[27] [23]
5. **Timing/tier integration** (if applicable)
   - If the experience declares timing indices (e.g., CSI bands), CI runs the timing safety checks defined for derived timing signals to ensure indices stay within declared timing caps for the tier.[22] [23]

Only experience contracts that pass all of these checks are merged into the library, guaranteeing that at runtime `H.Run`, `H.Director`, and `H.Metrics` can safely interpret them as envelopes that only *narrow* global/tier caps and never contradict the safety spine.[25] [27] [23]

If you want, next step could be drafting a concrete, GitHub-ready `experience-contract.v1.json` skeleton (no citations) for Constellation-Contracts plus a small CI pseudo-workflow that wires it into the existing spine and guardrail validators.

⁂

`metric_bands` and `invariant_requirements` specified in `Constellation-Contracts`? |

11	`[TECH] [DESIGN]` Define the interface between `Orchestrator` and `Codebase-of-Death`. What data is passed when a session is finalized? Is it a full history dump, a summary report, or a reference to a stored log file?
12	`[TECH] [DESIGN]` How does the `Spectral-Foundry` component query the `Orchestrator` to retrieve a list of valid `Atrocity-Seeds` for a given `runContract`? What authentication or authorization mechanism is used for this communication?
13	`[TECH] [DESIGN]` What is the data structure for passing serialized `History` objects between modules, particularly from `H.Selector` to `H.Director` after a node is chosen?
14	`[TECH] [DESIGN]` How is the `History` object versioned to ensure backward compatibility when new types of events or nodes are introduced? What is the upgrade/downgrade protocol?
15	`[TECH] [DESIGN]` What is the performance overhead of serializing/deserializing the `History` object, and what caching strategies should be employed to mitigate it during active gameplay?

Constellation-Contracts treats `metric_bands` and `invariant_requirements` as first-class, spine-aligned objects: every contract or policy that can shape runtime behavior exposes explicit numeric bands for invariants (CIC, DET,

SHCI, etc.) and metrics (UEC, EMD, STCI, CDL, ARR), and CI enforces that these bands are subsets of the canonical spine schemas.[29] [30]

Metric bands and invariant requirements

Within Constellation-Contracts, the canonical lists of invariants and metrics live in `invariants.v1.json` (CIC, MDI, AOS, RRM, FCF, SPR, RWF, DET, HVF, LSG, SHCI) and `entertainment-metrics.v1.json` (UEC, EMD, STCI, CDL, ARR). Any schema that needs to constrain world physics or player experience—run contracts, director personas, selector patterns, tier routing, ethics guardrails—declares either `invariantBands`/`invariantTargets` and `metricTargets`/`metricBands` using these keys, with numeric bands enforced to be within the spine ranges.[31] [30] [29]

The `run-contract.v1.json` skeleton shows the canonical shape: a `metricsEnvelope` block with `invariantTargets` (object keyed by invariant abbreviation, numeric values) and `metricTargets` (object keyed by metric abbreviation, numeric values), plus a `styleEnvelope.metricBands` object where each metric is a `[min, max]` array. Director persona contracts use `invariantBands` and `metricTargets` with the same key sets, and age-tier routing tables specify `maxInvariants` per tier (e.g., max CIC, AOS, DET, SHCI) to cap what any routed session may request.[30] [29]

CI contains a reusable ref-check that loads the canonical invariants and metrics schemas, then scans candidate schemas (selector rules/patterns, director personas, etc.) for keys under `invariantBands`, `metricTargets`, `historyCoupling`, or `entertainmentTargets`, failing any file that invents non-spine keys. This guarantees that all `metric_bands` and `invariant_requirements` across Constellation-Contracts are expressed in the same vocabulary and can be joined in telemetry.[31] [30]

11. Orchestrator ↔ Codebase-of-Death interface at session finalize

The Orchestrator's job is to govern artifacts and telemetry across the constellation; Codebase-of-Death is the engine/implementation repo that runs Lua and Godot/Unreal bridges. When a gameplay or AI-chat session is finalized, the interface between them is *not* a raw text dump; it is a structured reference layer plus high-level envelopes:[30] [31]

- The engine (Codebase-of-Death) writes NDJSON telemetry locally or to private infra conforming to schemas such as `run-session-metrics-v1.json`, `session-metrics-envelope-v1.json`, `history-selector-decision-event-v1.json`, and `ai-chat-session-route-v1.json`.[29] [30]
- Orchestrator receives or can query:
  - A **session summary record** (session metrics envelope) keyed by `sessionId`, including aggregate UEC/EMD/STCI/CDL/ARR, comfort outcome (`withinBand`, `tooIntense`, `tooFlat`), and join keys (regionId, seedId, descriptorId, runContractId, directorPersonaId).[31] [30]
  - A **route envelope** describing the sequence of contract IDs (moodContracts, eventContracts, runContract, experienceContract) and stage transitions, without narrative content.[29]
  - **References to stored logs**: file paths or object storage keys for raw NDJSON streams remain in vault/engine environments; Orchestrator and governance modules operate mostly on structured summaries and identifiers rather than ingesting entire histories into public repos.[30] [29]

Practically, this means the boundary is "summary and pointers": Codebase-of-Death owns raw logs; Orchestrator consumes envelope summaries and URIs or hashes that Dead-Ledger and governance modules can later verify.[31] [30]

12. Spectral-Foundry querying Orchestrator for Atrocity-Seeds

Spectral-Foundry sits in the Atrocity-Seeds domain and should not talk directly to vaults or registries; it queries Orchestrator, which fronts the registry and Dead-Ledger. The interaction looks like:[30]

- Spectral-Foundry sends a **typed query** describing the current `runContract` context: `runContractId`, `regionId`, `tier`, `routerStateId`, current invariant/metric targets, and optional tags (e.g., liminal tags, worldStyle).[29] [30]
- Orchestrator:
  - Uses registry descriptors and Atrocity-Seed manifests to find seeds whose invariant/metric bands are compatible with the provided `runContract` and router caps, as discussed in your previous questions.[29] [30]
  - Optionally requests Dead-Ledger decision tokens for any seeds or bundles that require entitlement, filtering out those lacking valid tokens.[29]

The response is a **list of seed descriptors**—IDs, `deadLedgerRef`, invariant/metric summaries, and tier/policy attributes—rather than full contract bodies. Authentication/authorization is handled by:[30] [29]

- Mutual trust within the constellation (service-to-service), plus
- Policy-level constraints:
  - Orchestrator enforces repo and tier constraints via `age-tier-routing-table.v1.json` and descriptor policy attributes (safetyTier, explicitness bands).[30]
  - Dead-Ledger capability envelopes (`repo-access-capability.v1.json`, `repo-access-proof-envelope.v1.json`) prove that Spectral-Foundry is allowed to see or act on certain vault seeds at a given tier.[30]

So the channel is "contract-aware RPC with Dead-Ledger-backed caps," not an unauthenticated list call.

13. History object structure between H.Selector and H.Director

The runtime design pushes toward a single canonical context/history object passed between modules instead of ad-hoc parameters. The recommended pattern is:[31] [29]

- A **context table** (first parameter in H. APIs), carrying:
  - `sessionId`, `runId`, `regionId`, `tileId`, optional `nodeId`, `patternId`, `directorPersonaId`, `experienceContractId`, `routerStateId`.[31]
  - Stage and difficulty: `stage` (OuterThreshold, Locus, Rupture, Fallout), `difficulty`.[29] [31]
- A **history/route state** object, separate from immediate context but referenced in it, capturing:
  - Nodes visited (node IDs, timestamps).
  - Patterns and contracts used (patternIds, eventContractIds, moodContractIds).
  - Metric snapshots per step (UEC, EMD, STCI, CDL, ARR, derived DET, etc.).
  - Selector decisions and director strategies (`allow`, `soften`, `imply`, `downshift`, `refuse`).[31] [29]

Between `H.Selector` and `H.Director` after a node is chosen, the data structure is typically:

```
local selectorResult = {
  patternId = ...,
  ruleSetId = ...,
  explicitnessBand = ...,
  allowedEventTypes = {...},
  forbiddenEventTypes = {...},
  invariantsSnapshot = { CIC = ..., AOS = ..., DET = ..., SHCI = ... },
  resolutionStrategy = "normal" -- or "deny", "reroute", etc.
}
```

plus an updated context carrying stage, nodeId, and metric snapshot. `H.Director` receives this selectorResult along with the context and metric/budget snapshots to compute a safety strategy and tone, and both modules log their decisions into standard telemetry envelopes (`history-selector-decision-event-v1.json`, director decision events) referencing the same join keys.[29] [31] [30]

14. History object versioning and upgrade/downgrade

Versioning is handled at the schema and envelope level rather than as an opaque binary blob:

- Each telemetry schema (`history-selector-decision-event-v1.json`, `session-metrics-envelope-v1.json`, `ai-chat-session-route-v1.json`) carries an explicit `schemaRef` (e.g., `history-selector-decision-event.v1`) and is treated as immutable once shipped.[31] [30]
- New event or node types are introduced by:
  - Adding new optional fields to v1 (backwards compatible) or
  - Defining a v2 schema and teaching Orchestrator and analysis tools to accept both and project them into a stable "analysis view."[30]

Upgrade/downgrade protocol is:

- **Upgrade at ingestion**: when Orchestrator or tooling sees an older history record, it applies a projection function to fill missing fields with defaults or derive them from existing data (e.g., new stage enums inferred

from DET trajectory or runContract curves).[31] [30]

- **Downgrade for legacy consumers**: if a newer history object is passed to an older module, callers should either:
  - Use a compatibility adapter that strips unknown fields and maps new enums onto legacy ones, or
  - Reject with a clear error if required fields are missing from the older schema's perspective.[30]

Because the core join keys (sessionId, regionId, seedId, descriptorId, runContractId) and primary metrics/invariants are stable, telemetry and history objects can evolve by adding fields without breaking cross-module joins.[31] [30]

15. Performance overhead and caching for History serialization

The design assumes JSON/NDJSON serialization of history and metrics for telemetry, with Lua and engine code holding richer in-memory structures. To keep overhead acceptable during active gameplay:[29] [31]

- **In-memory primary, serialized secondary**
  - Modules like `H.Run`, `H.Selector`, and `H.Director` pass history/state via in-memory Lua tables (and engine structures for node graphs).[29]
  - Serialization to NDJSON is batched or buffered—e.g., one history-selector decision event per turn, but written via a lightweight logging layer that flushes to disk or a queue asynchronously.[29] [31]
- **Caching strategies**
  - H.Run caches run-level difficulty bands and metric targets via helpers like `H.Run.getDifficultyBands(runId, difficultyName)` to avoid reparsing runContracts from JSON on every step.[31] [29]
  - Node graphs use adjacency lists built once at load time; H.Node scoring iterates neighbors without restructuring the graph per step.[31]
  - Persona inventory and Dead-Ledger decision tokens are cached per session in the shared context so selectors and directors don't repeat lookups.[29] [31]

The main overhead comes from writing NDJSON and computing metric updates, not from passing history between modules, and the system mitigates this by treating serialization as an append-only, batched operation while keeping the high-frequency path strictly in-memory.[29] [31]

⁂

| 16 | `[TECH] [DESIGN]` How will the Lua modules act as a bridge between the scripting environment and the C++ core logic of `H.*` APIs? What is the strategy for managing memory and preventing leaks across this boundary? |

17	`[TECH] [DESIGN]` Define the Lua API surface for interacting with `H.Selector`. How does a designer write a custom selection script in Lua that can access current metrics, history, and director persona?
18	`[TECH] [DESIGN]` How will the Lua environment handle errors originating from scripts executed by `H.Selector` or `H.Director`? Will these errors crash the game or be caught and reported to a debugging console?
19	`[TECH] [DESIGN]` What is the process for registering new Lua modules at runtime? How does the engine discover and load them without requiring a restart?
20	`[TECH] [DESIGN]` How should the `image-helper` behavior be exposed through the Lua API? Can it be invoked with parameters to control its style or intensity based on the current session state?

Lua sits on top of a C++ H. core as a thin, governed orchestration layer: native code owns memory, invariants, and I/O, while Lua modules call well-defined C APIs, receive plain tables, and return small envelopes so engines like Death-Engine, Godot, and Unreal can all share the same horror logic.[33] [34]

16. Lua ↔ C++ bridge and memory

The unified H. API spec already assumes "H." is implemented in Lua, with engines talking to it through a thin native bridge rather than embedding H. logic directly in C++ or GDScript. The recommended pattern is to embed a single Lua VM in the engine core (or per game instance), load the shared `hpcruntime` / H.* modules into it, then expose a narrow set of C ABI bridge functions like `CallHRunInitSession`, `CallHRunNextTurn`, `CallHSelectorSelectPattern`, and `CallHNodeDescribe` that marshal POD structs or JSON blobs in and out.[34] [33]

On the C++ side, you keep ownership of heavy resources (contracts, registries, buffers) and only push transient tables into Lua; Lua code never opens files or reads raw JSON for contracts, it always calls H.* loaders that are backed by native code or pre-decoded data. Memory safety is enforced by keeping the Lua surface allocation-light (short-lived tables per call, no long-lived references into engine buffers) and by using a standard `ok/data/error` envelope for every H. call so C++ can reliably clean up and early-out on failure rather than leaving half-initialized state around.[33] [34]

Garbage collection and leak prevention follow from this split: C++ owns large, long-lived allocations and explicitly frees them when sessions end, while the Lua VM's GC handles small tables built per call; CI and linters in Codebase-of-Death also forbid Lua from using `io` or custom allocators, so Lua cannot accidentally hold onto raw engine resources.[34] [33]

17. Lua API for H.Selector and designer scripts

The H. runtime spec already defines `H.Selector.selectPattern(sessionId, regionId, context)` as the core selection API, returning a selector result with pattern ID, invariants snapshot, and optional `deadLedgerRef`. Designers don't call raw selectors with dozens of parameters; instead, the engine provides them a canonical context table like:[34]

```
local ctx = {
  sessionId = sess_id,
  runId = run_id,
  regionId = reg_id,
  tileId = tile_id,
  nodeId = current_node_id,
  patternId = current_pattern_id,      -- optional
  directorPersonaId = dir_id,          -- optional
  experienceContractId = exp_id,       -- optional
  stage = "Outer",                    -- run stage
  difficulty = "Standard",            -- from runContract
  routerStateId = rtr_id,
}
```

and then calls `H.Selector.selectPattern(ctx, selectorHints)` internally. A designer writing a custom selection script in Lua can rely on helpers like `H.Metrics.snapshot(sessionId)` for current UEC/EMD/STCI/CDL/ARR, `H.Invariants.sample(regionId, tileId)` for CIC/MDI/AOS/SHCI/LSG, and `H.Director.loadPersona(sessionId)` for the current persona envelope and style bands.[33] [34]

Within a custom selector module, the flow is: normalize the incoming context, query metrics and invariants via H.Metrics and H.Invariants, optionally read `context.directorPersonaId` and `context.experienceContractId` to bias pattern scoring, and then return a small result table with pattern ID, explicitness band, allowed/forbidden event types, and an invariants snapshot, leaving all raw history and contract loading to H.* helpers.[33] [34]

18. Error handling for H.Selector / H.Director Lua scripts

All H. functions, including selector and director calls, use a standard envelope shape like `ok`, `data`, and `error { code, message, details }`, so native code and AI-Chat surfaces never see raw Lua stack traces. Lua modules are expected to catch internal errors and convert them into structured error codes from a shared `h-error-codes.v1.json` registry (e.g., `SESSIONNOTFOUND`, `PATTERNNOTFOUND`, `CONSENTINCOMPATIBLE`) rather than throwing; CI enforces that only codes declared in this schema are used.[34] [33]

At the engine boundary, C++/Godot/Unreal call into Lua using `pcall` or equivalent, and any thrown Lua error is treated as a single `ok=false` result with a generic internal error code that can be logged to a debug console but not crash the game. For development, a separate `H.Debug.logDecision(context, details)` helper records richer diagnostics to local logs, while production telemetry surfaces only structured decision events and error codes, ensuring that script errors are visible to developers but never propagate as hard failures during gameplay.[33]

19. Registering new Lua modules at runtime

Codebase-of-Death treats Lua as a scriptable layer but still governed by CI and repo manifests; new H.* modules are discovered via `require` and a small whitelist of allowed namespaces per repo, defined in `repo-manifest.hpc.repo.json`. At runtime, the engine's Lua bridge (`hpcruntime`) exposes a `require` cache and can load additional modules by updating search paths and calling `require` on newly available scripts, which allows hot-loading in development builds without engine restarts.[33]

The manifest-driven approach means that engines only expect specific H. namespaces (e.g., `H.Selector`, `H.Node`, `H.Run`, `H.Audio`, `H.ImageHelper`), and any new module intended for runtime must be added to the manifest and pass the Lua sandbox linter that checks allowed imports and forbids raw JSON access. Tools like Godot bridges call stable entrypoints (e.g., `H.Run.initSession`, `H.Run.nextTurn`) and do not need to be rebuilt when internal H. modules are extended, as long as the public API stays within the documented surface.[34] [33]

20. Exposing image-helper through Lua and tying it to session state

The research already standardizes `H.ImageHelper.describe(context, invariants, metrics, inventory)` as the canonical Lua interface for AI-Chat and engines: it returns text-only prompts plus hidden clue tags and suggested metric deltas, keeping actual image generation out of Tier-1 repos. The `context` argument includes run and persona fields such as `runId`, `sessionId`, `regionId`, `tileId`, `nodeId`, `eventStage`, `difficulty`, and `directorPersonaId`, while `invariants` and `metrics` come from `H.Invariants.sample` and `H.Metrics.snapshot`.[33]

Stylistic and intensity control is driven by policy and difficulty: an `imageHelper` block in `run-contract.v1.json` defines allowed style packs, maximum images per run, and a safety profile, and a separate `image-helper-policy.v1.json` maps difficulty levels and hiddenTags to metric contributions and constraints on CDL/ARR. Lua callers like `H.Run` or engine bridges can therefore invoke `H.ImageHelper.describe` with current `context`, `invariants`, `metrics`, and active persona inventory; the helper reads policy to adjust how concrete or abstract prompts should be, how many symbols to include, and how much to move UEC/EMD/CDL/ARR if players engage, all without exposing raw image data in the core repos.[34] [33]

⁂

| 21 | `[TECH] [DESIGN]` How will the `runContracts` schema evolve over time? What is the policy for adding new fields versus deprecating old ones to maintain compatibility with existing sessions and tools? |

22	`[TECH] [DESIGN]` What is the mechanism for loading and unloading `Atrocity-Seeds` dynamically during a session? Should this be tied to specific narrative nodes or handled globally by the `Orchestrator`?
23	`[TECH] [DESIGN]` How does the `Orchestrator` manage the lifecycle of a `session` from start to finish, including initialization, periodic saves, and final cleanup?
24	`[TECH] [DESIGN]` What is the format for telemetry data emitted by the `H.*` modules? How is this data structured to be easily ingested by an external analytics platform for post-mortem analysis?
25	`[TECH] [DESIGN]` How will the system handle updates to `Constellation-Contracts` while a live session is running? Should the system be resilient to minor changes, or must it be restarted?

Run contracts evolve by adding optional, spine-aligned envelopes (metrics, safety, imageHelper, difficulty) while keeping `run-contract.v1` stable for existing tools; new behavior comes from new policy contracts and schemas rather than breaking existing fields, and deprecation is handled by adding v2 schemas and upgrade adapters instead of mutating v1.[35] [36]

21. Evolving runContracts over time

The current `run-contract.v1.json` skeleton already encodes the intended evolution pattern: a fixed core (id, schemaRef, objectKind, tier, phase, regionRef, seedRef, personaContractRef) and a set of envelope blocks (`styleEnvelope`, `budgetEnvelope`, `safetyEnvelope`, `historySelectorEnvelope`, `metricsEnvelope`, `imageHelper`, `prismMeta`, `deadLedgerRef`). The policy is to keep this shape backward-compatible by only adding new optional properties within existing envelopes (or adding new optional top-level envelopes), never by changing required fields or semantics in place; when incompatible changes are needed, a `run-contract.v2.json` schema is introduced alongside v1 and CHATDIRECTOR learns to route and validate both.[36] [35]

Compatibility is enforced via Constellation-Contracts CI: `validateinvariants.rs` ensures new fields still reference the canonical invariant and metric spines, repo manifests pin allowed schemaRefs, and authoring skeleton generators return pre-validated v1 shapes that AI-Chat can only fill within given bands. For live tools and sessions, the usual pattern is "upgrade at load": when a runtime loader sees a v1 runContract and newer modules expect v2 features (say, richer difficulty or image rules), they use documented defaults or small adapter functions, while older modules ignore any fields they do not know, keeping existing sessions functional until you choose to migrate content.[35] [36]

22. Dynamic loading of Atrocity-Seeds during a session

Atrocity-Seeds themselves live as contracts and bundles in their own repos, but activation in a session is governed by Orchestrator, Dead-Ledger, and H.Selector/H.Run rather than ad-hoc script calls. The basic mechanism is: At session start, the selected `runContract` and region define eligible invariant/metric envelopes; Orchestrator (and Spectral-Foundry) pre-query the registry and Dead-Ledger to obtain a pool of allowed seeds plus decision tokens, which are attached to the session context as `deadLedgerDecisionToken` and seed descriptors.[36] [35]

During play, dynamic loading and unloading is tied to **narrative nodes and Haunt Vector fields**, not global switches: H.Selector or H.Node, operating on a given region/tile/node with high CIC/SHCI/LSG, can choose a pattern whose contract references a particular Atrocity-Seed and only then ask Policy.DeadLedger (via Orchestrator) for permission to activate that seed. If entitlement is granted, the seed's invariant bundle becomes active for that node and nearby tiles (as a local modifier to HVF/SHCI), and H.Metrics/H.Telemetry log which seed was active when; when the player leaves that liminal region or a prescribed number of beats elapse, selectors simply stop returning patterns tied to that seed, effectively unloading it without needing global teardown logic.[35] [36]

23. Orchestrator session lifecycle

Orchestrator is the governance and routing brain for runs: it wires consent, routing, contracts, and telemetry from session start through cleanup. A typical lifecycle is:[36] [35]

1. **Initialization**
   Orchestrator receives a session request (AI-Chat or engine) that includes account, region, experience or run preset, and target tier. It binds a consent profile via H.Consent, resolves tier caps via the Age-Gated Router, chooses or validates a `runContract` and persona, and issues an initialization envelope for the engine (or H.Run), including sessionId, runContractId, personaContractId, and routerStateId.[35] [36]
2. **Active session with periodic saves**
   The engine (via H.Run / H.*) owns fine-grained steps, but Orchestrator expects regular telemetry: consent metrics, budget events, history-selector decisions, and route segments (contract IDs plus UEC/EMD/STCI/CDL/ARR snapshots). Periodic "session metrics envelopes" and "run-session metrics" aggregates are written as NDJSON, and Orchestrator tracks them via join keys (sessionId, runId, descriptorId, routerStateId) while treating raw logs as implementation-side data.[36] [35]
3. **Finalization and cleanup**
   On session end—natural completion, user exit, or safety halt—H.Consent and H.Budget emit final metrics envelopes; H.Run or engine code flush outstanding selector and safety events; and Orchestrator composes a session route envelope (AI-Chat profile, sequence of contract IDs, stages, metrics curves, comfortOutcome). Orchestrator then marks the session closed, stores references to all telemetry streams (paths or object keys) plus derived analytics keys, and invalidates any temporary capabilities or Dead-Ledger tokens associated with the session, ensuring no further content is routed under that sessionId.[35] [36]

24. Telemetry format from H.* modules

Telemetry is NDJSON-based, schema-first, and join-key heavy so external analytics can reconstruct behavior without seeing text or vault data. Each H.* module emits records conforming to specific schemas in Constellation-Contracts, for example:[36] [35]

- **Consent and budget**: `consent-session-metrics-v1.json` and `budget-consumption-event-v1.json`, with top-level fields like `sessionId`, `consentTier`, `routerStateId`, `budgetRemaining`, modality breakdowns, and timestamps.[35]
- **Run and session metrics**: `run-session-metrics-v1.json` and `session-metrics-envelope-v1.json`, including `sessionId`, `runContractId`, `directorPersonaId`, `comfortPolicyRef`, segment-level UEC/EMD/STCI/CDL/ARR time series, and a comfortOutcome enum.[36] [35]

- **Selector and director decisions**: `history-selector-decision-event-v1.json` plus a director-decision schema, carrying `selectorPatternId`, `ruleCatalogId/hash`, node IDs, metric snapshots before/after, directorStrategy, and links to budget and safety events.[36]

All records share an identity layer: `sessionId`, `regionId`, `seedId` or `descriptorId`, `consentTier`, `routerStateId`, and often `selectorPatternId` or `runContractId`, making them trivially joinable in external analytics. Because payloads are strictly structured JSON with enums and validated numeric ranges, any external platform can treat them as append-only event streams, denormalized into tables keyed on session/run, to drive dashboards and post-mortems without ingesting model outputs or narrative text.[35] [36]

25. Handling live updates to Constellation-Contracts

Constellation-Contracts is treated as the "physics and policy spine," so live sessions must be shielded from sudden breaking changes while still allowing governance updates over time. The core strategies are:[35] [36]

- **Version pinning and manifest routing**: Run contracts, personas, selector rules, and policies all carry explicit `schemaRef` and version identifiers, and Orchestrator records which versions were in effect when a session started; that session continues to use those particular versions (or cached projections) for its lifetime, even if newer schemas or contracts are merged.[36] [35]
- **Backward-compatible evolution**: New invariants, metrics, or policy knobs are introduced by adding optional fields or new contract families, with CI jobs (`invariants-metrics-refcheck`, safety invariant validators) ensuring they only reference the canonical spines. Breaking changes (e.g., tighter SAF physics) are rolled out via new schemaRefs and promotion workflows, with Orchestrator only assigning them to *new* sessions after governance approval.[35] [36]
- **Runtime resilience vs restart**: For minor, backward-compatible updates (additional patterns, new director profiles, narrower metric bands within existing caps), the system is expected to be resilient: long-running services can reload catalog hashes or contract caches between sessions without restarts, while in-flight sessions keep their previously bound runContracts and selector catalogs. For major changes that affect engine wiring or require new Lua surfaces, a rolling restart of engine services is acceptable, but Constellation-Contracts ensures the on-disk schemas and manifests remain compatible with historical telemetry so analysis and replay tools never break.[36] [35]

⁂

| 26 | `[TECH] [DESIGN]` What is the storage mechanism for the `Dead-Ledger`? Is it a flat file, a database, or an in-memory structure with periodic disk persistence? |

27	`[TECH] [DESIGN]` How is the `Dead-Ledger` queried efficiently? What indexes or data structures are needed to quickly find records related to a specific `session_id` or `player_id`?
28	`[TECH] [DESIGN]` What is the retention policy for the `Dead-Ledger`? How long are records kept, and what is the process for archiving or purging old data?
29	`[TECH] [DESIGN]` How will the `Spectral-Foundry` handle conflicts when multiple `Atrocity-Seeds` provide assets with the same name or identifier? What is the resolution strategy (e.g., priority, error, merge)?
30	`[TECH] [DESIGN]` What is the security model for accessing and modifying the `Dead-Ledger`? Who or what has write permissions, and how is tampering prevented?

Dead-Ledger is a governed, append-only spectral ledger stored as structured NDJSON event streams (per repo and environment), backed by a database or object store, with a small in-memory cache and strict client APIs; engines never see raw files and only talk to it via `Policy.DeadLedger` and H. helpers.[37] [38]

26. Dead-Ledger storage mechanism

Dead-Ledger is treated as a protocol and telemetry spine, not as an ad-hoc in-memory table. Its entries (bundle attestations, agent attestations, spectral seed proofs, incident reports) are modeled as JSON contracts such as `ledger-entry-bundle-attestation.v1.json` and related proof envelopes, and are written as NDJSON events keyed by `deadLedgerRef` and `decisionTokenId`. The compiled TODO report explicitly calls out "Dead-Ledger proof envelope verification scripts" and positions Dead-Ledger in its own repo (`HorrorPlace-Dead-Ledger-Network`), which implies a persistent, queryable store (e.g., database or log + index) with CI hooks rather than a purely in-memory structure.[38] [37]

At runtime, engines do not directly read ledger files; they call the `Policy.DeadLedger` client defined in the unified H. runtime API (`horror-engine-lua-api-v1.md`). That client talks to the underlying ledger service, which may cache recent `decisionTokenId`→cap decisions in memory for speed, but durability and auditability are guaranteed by the NDJSON streams and their schema-validated storage.[37] [38]

27. Efficient querying and indexing

Dead-Ledger is optimized around a small set of join keys and IDs that appear everywhere in the constellation: `sessionId`, `regionId`, `runContractId`, `deadLedgerRef`, `decisionTokenId`, and sometimes `agentDeadLedgerRef`/`seedDeadLedgerRef`. Telemetry and policy calls always write and read using these keys, and schemas require them at top level so external analytics and ledger validators can join events cheaply.[38] [37]

For efficient lookups, the ledger backend should maintain indexes at least on:

- `deadLedgerRef` and `decisionTokenId` (to validate tokens quickly in `Policy.DeadLedger.checkToken` and similar helpers).[37] [38]
- `sessionId` and `agentId`/`playerId` (for audit trails, incident reconstruction, and entitlement usage analysis).[38]

The runtime access pattern is deliberately narrow: game code calls `Policy.DeadLedger.requestBundleAccess(deadLedgerRef, context)` or `requestSpectralSeed(...)` and receives a small envelope (`decision`, `safetyTier`, `intensityBand`, `decisionTokenId`, `expiresAt`). That envelope is then attached to the H. context as `deadLedgerDecisionToken` and propagated through selectors and directors, so subsequent modules only check a short token rather than re-querying the ledger directly. This "front-loaded token, cached in context" design drastically reduces query load while preserving server-side validation and auditability.[37] [38]

28. Retention and archiving policy

Dead-Ledger sits in the safety and governance spine with ABSOLUTE invariants tied to incident logging and proof attestation, so its records are intentionally long-lived. The safety physics spec and compiled TODOs describe "Dead-Ledger incident notification" and "attestation" for ABSOLUTE violations, and treat those as part of governance evidence, which implies they must not be short-lived or silently purged.[38]

A practical policy consistent with the framework is:

- **Hot window** (e.g., 30–90 days): full, online NDJSON streams indexed by `sessionId`, `deadLedgerRef`, `decisionTokenId`, and incident category; needed for rapid debugging and early tuning of invariants.
- **Warm archive** (e.g., 1–2 years): compacted or partitioned storage (object store buckets by month, or database partitions) retaining all ledger entries but moving them out of the primary low-latency tables.[38]
- **Cold archive / compliance**: depending on legal needs, ABSOLUTE-violation and proof records may be retained longer (or indefinitely) in encrypted cold storage, since they are central to governance and audit; non-critical, budget-gated entries can be aggregated.[38]

Purging is handled by offline jobs that roll up or delete old entries while preserving referential integrity with telemetry and promotion logs (Constellation-Contracts governance). Because `deadLedgerRef` and proof envelopes are referenced from safety and selector schemas, any archival process must keep those IDs resolvable for as long as corresponding telemetry is retained, so that post-mortem analyses remain consistent.[38]

29. Spectral-Foundry conflicts for Atrocity-Seed assets

Spectral-Foundry lives on top of Atrocity-Seed contracts and Dead-Ledger entitlements and must treat spectral assets as governed, not arbitrary. When multiple seeds offer assets with the same name or identifier, the resolution strategy has to preserve auditability and safety:[38]

1. **Namespacing and manifest keys**
   Atrocity-Seed repos already use contract IDs and `deadLedgerRef` rather than bare filenames; Spectral-Foundry should treat each asset as `seedId + assetId` and only expose collision-free, namespaced IDs upstream. Seed manifests can declare a local `assetNamespace` or explicit `assetId` keys to avoid accidental overlaps.[38]
2. **Priority and capability-driven resolution**
   If two seeds still map to a logically similar asset (e.g., same semantic tag), Spectral-Foundry can prefer one based on an explicit policy: tier, safety band, or priority encoded in the Atrocity-Seed contract or its

Dead-Ledger proof. For example, a standard-tier seed's assets win over a research-tier seed in public runs, or a seed with stricter DET caps is favored by default.[38]

3. **Hard conflicts as errors, not silent merges**
   The design philosophy across the constellation is "schema-first, no silent drift": safety invariants and selector rules do not auto-merge; they reject inconsistent states. Spectral-Foundry should follow the same rule: if two seeds declare the exact same asset identifier in the same context without clear priority, the registry or build must fail with a structured error, prompting a design correction, rather than guessing or merging content.[38]

In practice, this yields a simple rule: Atrocity-Seeds are merged at the contract and manifest level under strict namespacing and policy priorities, and any ambiguous duplicate identifiers are surfaced as errors during manifest compilation or CI, not resolved at runtime.[38]

30. Security model and tamper resistance

Security for Dead-Ledger is multi-layered: narrow clients, immutable physics invariants, CI enforcement, and cryptographic proofs.[37] [38]

Only the following actors have **write** access:

- The Dead-Ledger service itself, which appends verified entries (bundle attestations, proof validations, ABSOLUTE-violation incidents) that conform to ledger schemas.[38]
- Governance pipelines (promotion workflows) that create or update attestation bundles under strict CI and promotion rules; these are not runtime game clients but controlled, audited processes.[38]

Engines, AI-Chat, and H. Lua code never write directly to the ledger; they only call `Policy.DeadLedger` functions (`requestBundleAccess`, `requestAgentAccess`, `requestSpectralSeed`, etc.) which in turn record decisions and proofs. Static analysis in Codebase-of-Death already enforces that runtime Lua cannot perform arbitrary I/O into schemas or registry paths and can only access invariants and contracts through H. and `Policy.DeadLedger`. The same doctrine extends to Dead-Ledger: no direct file I/O or raw JSON writes to ledger paths, only the sanctioned client.[37] [38]

Tamper resistance is achieved by:

- **Schema and CI enforcement**: ledger entries must validate against Constellation-Contracts schemas; any change to SAF invariants or ledger behavior runs through reusable CI workflows in HorrorPlace-Orchestrator that reject unsafe threshold changes or attempts to loosen ABSOLUTE invariants.[38]
- **Dead-Ledger proofs and flags**: ABSOLUTE violations are required to raise incidents with a `deadLedgerFlag` in the safety physics schema, tying them to attestation and audit flows in the ledger repo.[38]
- **Opaque references and tokens**: runtime code only sees `deadLedgerRef` and synthetically generated `decisionTokenId` with expiry; it cannot forge or modify proofs because verification happens server-side (in the ledger client / service), and tokens are checked again whenever used in selectors or entitlement validation.[37] [38]

This yields a clear security posture: only governed services can write to Dead-Ledger, all runtime modules go through a small, audited client, any attempt to bypass that is caught by static analysis and CI, and ABSOLUTE-class events always leave a signed, immutable trace that can be correlated with telemetry for forensic and compliance purposes.[37] [38]

⁂

| 31 | `[TECH] [DESIGN]` How does the `Orchestrator` authenticate itself to the various backend services it needs to interact with (e.g., telemetry servers, asset stores)? |

32	`[TECH] [DESIGN]` What is the fallback behavior if a required `Atrocity-Seed` cannot be found or fails to load? Does the `H.Selector` attempt to find a substitute, or does the session fail gracefully?
33	`[TECH] [DESIGN]` How is the `History` object persisted between sessions to enable replayability? What information is essential to save for a true replay?
34	`[TECH] [DESIGN]` What is the maximum size limit for a `History` object before it becomes a performance bottleneck, and what pruning or summarization strategies should be implemented?

| 35 | `[TECH] [DESIGN]` How does the system handle network latency when fetching assets from `Spectral-Foundry` in an online multiplayer context? |

Orchestrator authenticates to backends using governed capabilities and Dead-Ledger–backed proofs rather than ad-hoc API keys, Atrocity-Seed failures always degrade to safe substitutes or non-horror beats instead of crashing runs, History is persisted as a compact, contract- and metric-driven route that can fully reconstruct decisions without storing raw text, and Spectral-Foundry is expected to hide latency through caching and scheduling so multiplayer clients never block on seed assets.[39] [40]

31. Orchestrator authentication to backends

Within the constellation, Orchestrator is part of the governed runtime and must authenticate to other services (telemetry, registry, Spectral-Foundry, Dead-Ledger) using structured capabilities, not hardcoded secrets. Constellation-Contracts already defines "repo-access capability" and Dead-Ledger envelope patterns, and the H. API spec describes `Policy.DeadLedger` as the single, audited client for entitlement. Orchestrator should therefore hold:[40] [39]

- A **service identity** (e.g., `orchestrator-service-id`) with associated Dead-Ledger or ALN proofs that give it authority to write telemetry, read registry descriptors, and request bundle/seed access.[39]
- Short-lived **access tokens/capabilities**, issued by a central auth or capability service, which encode which repos and topics it may write to (telemetry buckets, asset catalogs, etc.) and which Dead-Ledger operations it may invoke.[39]

When Orchestrator calls telemetry sinks or asset/registry services, it sends this capability plus trace IDs, and those backends validate both the token and the schema of what is written (e.g., `session-metrics-envelope-v1.json`, `registry-events-v1.json`). For Dead-Ledger interactions, Orchestrator does not embed any private key material; it calls `Policy.DeadLedger.*` with `deadLedgerRef` and a small context, and that client performs proof verification and writes ledger entries, keeping the cryptographic surface centralized.[40] [39]

32. Fallback when Atrocity-Seed is missing or fails

The Atrocity-Seed and Dead-Ledger research explicitly recommends that selectors and directors **never** assume an attested bundle is available; they must treat entitlements as a gate and always have a lower-tier fallback. The pattern is:[40] [39]

1. When a pattern or node references a seed (`deadLedgerRef` on a run, node, or bundle), H.Selector or H.Node first asks `Policy.DeadLedger.requestBundleAccess`/`requestSpectralSeed` with the ref and a context, receiving a decision plus a `decisionTokenId` on success.[39] [40]
2. The `decisionTokenId` is attached to the canonical context table (`context.deadLedgerDecisionToken`) and passed into selectors and directors; only if this token is present and valid can the seed's assets be considered eligible.[40]

If a required Atrocity-Seed is missing in the registry, fails Dead-Ledger validation, or the entitlement call returns deny/expired, runtime must treat this as "bundle ineligible": selectors and nodes mark the bundle or node as unavailable and **downshift** rather than fail the run. In practice, H.Selector should search its catalog for alternate patterns at the same structural location (same stage, similar invariant/metric envelopes) that either:[39] [40]

- Use a different, valid seed, or
- Use only public, non-attested content stacks for that region.[40]

If no alternatives exist, the safety spine and runtime fallback policies say runs should drop to a non-horror or low-DET mode instead of hanging or emitting an error: AI-Chat is nudged into a safe persona and selectors favor cooldown or epilogue patterns, with clear in-world justification ("the locked archive is sealed; you only glimpse it from afar"). Telemetry logs that a Dead-Ledger entitlement failed and a fallback path was chosen, so designers can fix missing seeds or adjust patterns later.[39] [40]

33. Persisting History for replayability

The existing telemetry and route research gives you the outline of a **History object**: it is essentially a structured replay log, composed of session metrics, selector decisions, director decisions, budget events, and run-session summaries, all keyed by shared IDs (`sessionId`, `runContractId`, `selectorPatternId`, etc.). For true replay—being able to deterministically step through a past run—you must persist:[40] [39]

- The **identity layer**: `sessionId`, `runContractId`, `directorPersonaId`, `experienceContractId` if any, `regionId`, and any dungeon/run IDs.[39]
- The **decision stream**: ordered events for each turn, including history-selector decisions (`selectorPatternId`, rule catalog snapshot, node IDs), director safety decisions (strategy, comfort policy ref), and budget events.[39]
- The **metric snapshots**: pre- and post-step UEC, EMD, STCI, CDL, ARR, plus derived DET and band/budget flags (budget band, comfortOutcome).[39]
- The **branch choices**: which player options were taken (for interactive choices), as small, schematized IDs with associated intended metric deltas, not raw text.[40]

Because schema docs and telemetry already define NDJSON envelopes like `history-selector-decision-event-v1.json`, `budget-consumption-event-v1.json`, `run-session-metrics-v1.json`, you can reconstruct a "History" object by grouping and ordering these events per session. For machine-driven replay, you do not need to store model outputs or natural language; instead, you re-run the H. state machine (H.Run, H.Selector, H.Director, H.Safety) in **replay mode**, feeding it the same sequence of branch and selector decisions and letting it regenerate text from the same contracts and templates.[40] [39]

34. History size limits and pruning

The limiting factor for History is not raw bytes—NDJSON scales—but **in-memory complexity** during live sessions and the cost of analytics over huge event streams. While there is no hard numeric limit defined, the research suggests a layered approach:[39]

- **Fine-grained, per-turn events** are written to telemetry during the run (selector decisions, metrics snapshots, budget events) and do not need to be kept in RAM for the entire session; H.Run can maintain only the current context and a small, recent window (for SAF escalation windows, cooldown ratios, repetition checks).[40] [39]
- **Per-run aggregates** (run-session metrics) summarize the trajectory: difficulty, persona, completion status, branch types, and aggregate UEC/EMD/STCI/CDL/ARR curves, and are sufficient for many analyses.[39]

For History objects exposed to AI-Chat or engine tools, you should set practical caps such as:

- Only keep the last N turns' detailed data in the live runtime context (e.g., 50–100 beats), plus precomputed aggregates; older beats can be flushed to telemetry logs and referenced by ID only.[40]
- When building a "History snapshot" to send to AI-Chat, compress older segments into **segment summaries** (e.g., "Outer: DET from 1.0→3.0, CDL stable, ARR high; patterns A/B/C visited") and only include full detail for the most recent blocks relevant to current decisions.[39]

Pruning strategies are therefore:

- **Rolling window in memory** based on SAF escalation windows and repetition logic.[40] [39]
- **Segmentation** (Outer/Locus/Rupture/Fallout) with per-segment aggregates, using `session-segmentation-v1.json` and run-stage enums; the History object can store segments and recent steps, dropping fine detail for earlier segments.[39]
- **Compression/summarization** when serializing History for offline tools or cross-session adaptation: keep branch IDs, metric deltas, comfortOutcome, and key pattern IDs rather than every intermediate detail.[39]

If you find that History snapshots begin to cause performance issues (large JSON contexts to AI-Chat, heavy in-memory structures in H.Run), the fix is always to flow more through telemetry and recompute as needed from logs, rather than raising caps.[40] [39]

35. Network latency and Spectral-Foundry assets in multiplayer

The multi-engine research treats image and asset helpers as **advisory, not blocking**: H.ImageHelper and Spectral-Foundry are designed so text, metrics, and safety decisions proceed even if images or spectral assets arrive late or not at all. For online multiplayer, the core strategies are:[39]

- **Asynchronous planning**: game logic calls something like `H.Images.plan` / Spectral-Foundry planning APIs with the current context (runId, sessionId, region/tile invariants, metrics, persona), which returns a descriptor and hidden tags, not a blocking asset fetch. Engines then issue non-blocking asset requests against Spectral-Foundry based on that descriptor.[40] [39]
- **Local caching and prefetch**: Atrocity-Seed manifests can mark frequently used assets per region/stage; before or during a match, clients or edge services pre-warm caches for expected seeds and styles, so high-priority

assets are already local when needed. Selector patterns and runContracts know which regions/stages might request which packs, making prefetch deterministic.[39]

When latency occurs or an asset is unavailable, the fallback is to **gracefully degrade visuals**:

- Use lower-fidelity or placeholder assets that still respect safety and invariants (e.g., generic fog or silhouettes), driven by local invariant→render mappings (`H.Invariants.toRenderParams`), while waiting for Spectral-Foundry's richer outputs.[40] [39]
- Ensure that **gameplay decisions never depend on the presence of a specific asset**; they read invariants, metrics, and contract IDs, not file availability, so matches stay authoritative and deterministic across clients.[40]

Orchestrator and H.Run should treat Spectral-Foundry as helpful but non-critical: if a given call times out or an asset fails to arrive, the text and node progression still advance, and telemetry marks that an image or asset opportunity was missed. Multiplayer engines can synchronize on text and node choices (which are light and deterministic) and let each client fetch and show Spectral-Foundry assets opportunistically, ensuring that network jitter affects only atmospheric richness, not shared horror logic.[40] [39]

⁂

| 36 | `[TECH] [DESIGN]` What is the mechanism for hot-swapping or updating Lua modules without stopping the game server? |

37	`[TECH] [DESIGN]` How will the `DirectorPersona` be loaded and applied at the beginning of a session? Is it part of the initial `runContract` payload or loaded separately?
38	`[TECH] [DESIGN]` What is the exact data format for transmitting `node` definitions from `Atrocity-Seeds` to the `H.Selector`?
39	`[TECH] [DESIGN]` How does the system differentiate between a temporary failure (e.g., a missing asset) and a permanent one (e.g., a corrupted contract)?
40	`[TECH] [DESIGN]` What is the detailed procedure for integrating a new game engine (e.g., Death-Engine) into the existing binding and orchestration framework?

Lua hot-swapping is done via a narrow H. runtime surface and engine bridges that reload modules or contract tables behind stable function signatures, DirectorPersonas are treated as contracts that are loaded and intersected with consent/budget at session start, Atrocity-Seed nodes travel as sanitized contract-driven tables that H.Selector consumes without touching raw vault JSON, temporary vs permanent failures are separated by a shared error-code schema and retry hints, and integrating a new engine like Death-Engine means embedding Lua once and exposing only a thin set of H.* calls to native code or visual scripting.[41] [42]

36. Hot-swapping Lua modules

The constellation assumes Lua is the **single behavior authority** and engines see only a small H.* API surface, so "hot-swap" means updating Lua code or data contracts while keeping those H.* entrypoints stable. Codebase-of-Death already defines a pattern where engine bridges (Godot, Unreal, Death-Engine) call functions like `H.Run.start`, `H.Run.step`, `H.Selector.selectPattern`, and `H.Node.describe` through an embedded Lua VM, while the underlying Lua modules and contract-derived tables can be reloaded.[42] [41]

Mechanically, hot-swapping works by treating contracts and H.* modules as **data-driven** and reloadable: new JSON schemas or pattern catalogs are compiled into Lua tables (or loaded on demand), and a loader module replaces old tables with new ones behind stable accessors like `H.Contract.load...` or `H.Selector.loadRuleCatalog`. Engine bridges don't import Lua modules directly; they call a single umbrella module (e.g., `hpcruntime`) that re-exports H.* and can internally run a "reload contracts/rules" step between turns or via an admin command, so sessions see updated patterns/rules without restarting the process. CI and manifests enforce that only approved H.* namespaces are visible per repo; this keeps hot-swap controlled and ensures no engine script reaches into raw Lua state that might disappear mid-session.[41] [42]

37. Loading and applying DirectorPersona

Research around `H.Director.loadPersona` describes a small cache keyed by `sessionId` holding the **effective persona envelope**, which merges persona contracts, comfort policy, consent caps, and budget caps. At the beginning of a session, H.Run (or an equivalent orchestrator helper) takes the chosen `runContractId` and `personaContractId`, validates them via Constellation-Contracts, and calls `H.Director.loadPersona` to compute the effective envelopes (invariants, metric bands, comfort policy) for this session.[42] [41]

The `runContract` itself references persona metadata and comfort policy (e.g., `personaContractRef`, `comfortPolicyRef`), but actual persona loading is done via these H.* helpers, not by directly decoding JSON in engine code, which keeps the persona logic centralized. Once loaded, the persona envelope is cached per session and re-derived when consent or budget revision stamps change; this gives you cheap reads throughout the run, while still allowing live downshifts if consent is downgraded or budgets approach caps. DirectorPersona IDs are also threaded into selector and metrics APIs (e.g., optional `directorPersonaId` in selector context and telemetry), so persona choice is visible in logs and can be correlated with UEC/EMD/STCI/CDL/ARR trajectories.[41] [42]

38. Data format for nodes from Atrocity-Seeds to H.Selector

The Atrocity-Seed research and the node/selector docs push a strict doctrine: engine and H.* code **never** touch raw vault JSON; they only see sanitized, contract-shaped tables. Dungeon node contracts are loaded through `H.Contract.loaddungeonNodeContract, nodeId`, which returns a normalized node card including things like `role`, `topology.connections`, `liminalTags`, `mechanicHooks`, and invariant/metric bands. H.Node helpers then access nodes through `NodeContract.getConnections(nodeId)` and `NodeContract.describe(nodeId)`, which flatten out invariants and bands into a stable table shape suitable for scoring and description.[42] [41]

For Atrocity-Seeds specifically, the "node definitions" that H.Selector sees are effectively **pattern-aware node cards** derived from seed manifests and selector pattern configs: they contain node IDs, roles, tags, invariant snapshots (CIC, AOS, DET, SHCI, LSG, etc.), and possibly metric targets, but never raw seed content or vault-internal structures. These cards flow into a shared scoring kernel (`enginelibrary/horrorscoring.lua`), which H.Selector and H.Node both use to rank candidates based on liminal tags, mechanic hooks, SHCI, and alignment with entertainment targets, keeping behavior consistent across pattern-level and node-level selection. The physical transmission from Atrocity-Seed repos to H.Selector is via JSON contracts (seed manifests, node contracts, selector patterns) that are compiled into those Lua tables; H.Selector's public API still just takes IDs and context, not raw node JSON, which keeps GitHub-facing code safe.[41] [42]

39. Temporary vs permanent failures

All H.* modules are expected to use a **shared error-code schema** (e.g., `h-error-codes.v1.json`) that enumerates canonical error codes like `SESSIONNOTFOUND`, `PATTERNNOTFOUND`, `ENTITLEMENTEXPIRED`, and `CONTRACTCORRUPT`, along with suggested fallback actions (`retry`, `downshift`, `fail`). Temporary failures, such as missing or slow assets, are signaled with codes and details that imply **retry or degrade** (for example, `ASSETUNAVAILABLE_RETRYABLE`) and are handled by downshifting to placeholders, safer patterns, or non-horror beats while telemetry records the issue.[42] [41]

Permanent failures, such as schema-invalid contracts, corrupted Atrocity-Seed manifests, or Dead-Ledger proofs that fail verification, are signaled with codes like `CONTRACTCORRUPT` or `ENTITLEMENTDENIED`, where retry would be meaningless. In these cases, runtime must treat the affected content as ineligible and either route around it (select alternate patterns/nodes that don't depend on the bad contract) or gracefully end the run in a safe epilogue mode, never attempting to "fix" or reinterpret the corrupted data on the fly. CI complements this at build time by validating that any referenced contracts and seeds conform to schemas and reference only canonical invariants/metrics; corrupted or out-of-spine objects should never make it into a production route catalog, making permanent failures rare in live systems.[41] [42]

40. Integrating a new engine (Death-Engine) into bindings and orchestration

Engine integration is designed to be **minimal and consistent**: embed Lua once, load the H.* runtime, and expose a small set of bridge functions to the engine's own scripting or node system. For Death-Engine, Codebase-of-Death already sketches a C/Rust bridge that embeds the Lua VM, loads `horror-engine-lua-api-v1` and runtime modules, and exposes functions like `CallHRunStart`, `CallHRunStep`, `CallHSelectorSelectPattern`, and `CallHNodeDescribe` to Blueprint- or VisualScript-style systems.[42] [41]

The procedure is roughly:

1. **Embed Lua and load H.**: Death-Engine starts an embedded Lua runtime that initializes the H.* namespaces (`H.Run`, `H.Selector`, `H.Node`, `H.Safety`, `H.Invariants`, etc.), all wired to read contracts from Constellation-Contracts rather than raw files in engine projects.[41]
2. **Implement a thin native bridge**: a small native layer exposes only high-level H.* functions with simple POD or JSON arguments/returns, so Blueprints or other scripting languages never see Lua directly.[41]

3. **Bind engine-side "nodes" or functions**: designers work with nodes like `HRunStart`, `HRunStep`, `HSelectorSelectPattern`, which marshal game state (sessionId, regionId, tileId, context) into the canonical context table and call into Lua, then map results to audio, VFX, and gameplay.[42] [41]
4. **Enforce governance via CI and manifests**: repo manifests declare which H.* namespaces are allowed in Codebase-of-Death; CI linters ensure Death-Engine code and Lua scripts only import approved modules and never touch raw invariants or contracts.[41]

Because behavior is centralized in Lua and data contracts, adding Death-Engine (or any new engine) does not require rewriting horror logic, only implementing that narrow bridge and ensuring telemetry, error codes, and invariant queries are wired through H.* as specified.[42] [41]

⁂

Design Methodology & Metric Calibration

This section contains approximately 30 questions focused on the design methodology and the calibration of the horror experience. It explores how designers can effectively use the project's formalized invariants, metrics, and contracts to create compelling, terrifying, and repeatable experiences. The emphasis is on translating abstract horror theory into objective, measurable parameters that govern the `H.Director` and `H.Selector`.

ID	Question / Query
41	[DESIGN] What is the definitive definition of 'UEC' (Unsettling Event Count) and what specific in-game actions or events contribute to its value?
42	[DESIGN] What is the definitive definition of 'EMD' (Environmental Mismatch Density) and how is it calculated from the game world's geometry, lighting, and object placement?
43	[DESIGN] What is the definitive definition of 'STCI' (Spectral-Tension-Counter-Index) and what is its numerical range and update frequency?
44	[DESIGN] What is the definitive definition of 'CDL' (Corruption-Density-Level) and how does it relate to the SHCI invariant?
45	[DESIGN] What is the definitive definition of 'ARR' (Atrocity-Response-Rate) and what constitutes a valid player response that affects this metric?

UEC counts how many meaningful unsettling beats a player lives through, EMD measures how densely mysteries and environmental contradictions are packed into the experience, STCI tracks how sharply and how often the session alternates between safety and threat, CDL quantifies how saturated the player is with unresolved contradictions and cognitive dissonance, and ARR measures how many of the session's mysteries deliberately remain unresolved at structurally important moments.[43]

41. UEC – Uncertainty / Unsettling Engagement Coefficient

Within the current spine, UEC is treated as a **session-level "uncertainty engagement" metric** in the 0.0–1.0 range, not a raw counter. It represents how consistently the player is engaged with *unsettling* but legible situations: there are active questions and tension, but the experience still feels navigable rather than random.[43]

In practice, UEC is incremented by events and templates that: introduce new, graspable mysteries; escalate existing questions in a coherent way; or present choices whose consequences are unclear but interpretable. Narrative forks marked as "brave" or "curious," liminal node entries that expose fresh information, and images or clues with hiddenTags that the player notices are all sources of positive UEC deltas via their contract-encoded `deltaUEC` values. UEC goes down or stays flat when the experience becomes over-explained, purely mechanical, or when the player spends too long in "neutral" connective tissue with no meaningful uncertainty.[43]

42. EMD – Environmental Mystery / Mismatch Density

EMD is defined as **mystery density**: how many active mysteries and unresolved environmental anomalies are in play per segment, also normalized to 0.0–1.0. It is not "any weird object," but the density of *live* threads that a player is expected to track: unexplained symbols, contradictory logs, strange sounds with no clear source, liminal nodes that imply more than they explain.[43]

Design-wise, EMD is derived from contracts rather than raw geometry: node contracts, event contracts, and image-helper policies declare how many hiddenTags, anomalies, or clue symbols are present in a beat and how strongly they should push EMD up or down. A corridor with many visual oddities, conflicting recordings, or lore fragments will have higher EMD contributions than a straightforward hallway, even if they share geometry,

because the contracts encode more concurrent mysteries. Runs tuned for “comfortable mystery” keep EMD roughly 0.1–0.4; above about 0.5–0.6, especially when paired with high CDL, the system considers players at risk of cognitive overload.[43]

43. STCI – Safe–Threat Contrast Index

STCI is a **contrast index** that measures how strongly and how often the experience alternates between safe and threatening states, again as a 0.0–1.0 scalar. Low STCI means the session is flat (always safe or always threatening); mid ranges represent gentle oscillations; high STCI means sharp, noticeable swings between comfort and danger.[43]

It is updated once per turn or beat based on the DET contribution of the current event and its relation to neighboring beats: high-DET events after low-DET moods increase STCI, while long sequences of similar DET reduce it or keep it flat. Reference curves for 20–40-turn sessions target STCI around 0.3–0.5 at the start, with two or three peaks in the 0.7–0.8 range and cooldown periods bringing it back down, enforcing a sawtooth “rising and falling tension” pattern. Designers use STCI bands in runContracts and director personas to guarantee that Slowburn runs have smoother, longer waves while Short Shock runs hit higher STCI faster and with fewer cooldowns.[43]

44. CDL – Corruption / Cognitive Dissonance Level and SHCI

CDL is defined as **cognitive dissonance level**, quantifying how saturated the player is with unresolved contradictions and incompatible explanations. It is separate from EMD: EMD is “how many mysteries are present,” while CDL is “how far those mysteries have stretched the player’s belief without resolution.” [43]

CDL is driven by encounters that introduce conflicting evidence, unstable explanations, or revelations that break earlier assumptions, and it decays when the Director provides clarifications, resolutions, or stabilizing context. Numerically it is a 0.0–1.0 band used in runContracts, director personas, and comfort policies, and it is tightly tied to DET and SHCI via cross-metric rules: regions with high SHCI (strong spectral-history coupling) and high DET support higher CDL floors and ceilings because the world’s history is coherent enough that heavy, strange revelations still feel “real” rather than random noise. In contrast, high CDL in low-SHCI regions is treated as a design smell, and schemas and comfort policies limit CDL spans so players do not experience pure confusion; standard tiers aim to end most runs with CDL around 0.4–0.6 when ARR is high, so endings feel intriguingly unresolved but not incoherent.[43]

45. ARR – Atrocity Resolution / Response Rate

ARR is modeled as a **resolution ratio for atrocities and core mysteries**, again 0.0–1.0, where higher values mean more critical questions remain open. It is not a morality score; instead it is the fraction (often weighted) of major mysteries, atrocities, or traumatic events that remain unresolved or only partially resolved at structurally important nodes like climaxes and endings.[43]

Each runContract and node/event contract tags a set of “structural mysteries” and encodes, for each branch or ending, whether those mysteries are resolved, partially resolved, or deliberately left open; ARR is computed as a weighted sum over those statuses. A run with ARR ≈ 0.2–0.3 at the end feels like a “Closed Case,” while ARR ≈ 0.7–1.0 corresponds to “Unresolved Haunt” experiences where most questions remain open by design. Valid player responses that affect ARR are any choices or actions that progress or close a structural mystery: choosing a path that reveals the origin of a haunt, performing or refusing a ritual that clarifies what happened, or using interpretation tools to connect distant events; contracts for those forks carry `deltaARR` entries, and H.Director clamps them against persona and experience-contract ARR bands so endings stay within the intended ambiguity range.[43]

⁂

| 46 | `[DESIGN]` How should the 'bands' for UEC, EMD, STCI, CDL, and ARR be calibrated to create distinct tiers of horror intensity (e.g., 'Apprehension', 'Dread', 'Panic')? What playtesting methodologies are appropriate? |

47	`[DESIGN]` How do the `metric_bands` influence the `H.Director`'s decision-making process? Are they a hard ceiling, a weighted preference, or a probabilistic trigger?
48	`[DESIGN]` Can a single `experience_contract` define multiple, branching narrative paths, each associated with a different set of `metric_bands` and `director_personas`?
49	`[DESIGN]` How can designers use the `Comfort_Policies` to create dynamic pacing? For example, how might a 'Safe_Band' allow for exploration while a 'Tense_Band' forces linear progression?
50	`[DESIGN]` What is the process for defining and authoring a new `runContract`? What tools or templates are provided to ensure adherence to the `Constellation-Contracts` schema?

For these five questions, the project already has a concrete design pass: bands are numeric "lanes" that define named horror tiers and pacing, `metric_bands` act as soft targets with hard-cap behavior at the edges, a single `experience_contract` absolutely can own multiple branches with different bands/personas, `Comfort_Policies` are the pacing governor that flip between exploratory and constraining modes, and `runContract` is a first-class bundle type with schema + explainer docs in Constellation-Contracts.[45]

46. Calibrating bands for Apprehension / Dread / Panic

The existing pass distinguishes Slowburn Dread vs Short Shock by UEC, EMD, STCI, CDL, ARR bands; you can treat those as concrete "Dread" and "Panic" presets and derive an "Apprehension" tier below them. For example, Slowburn Dread (20–40 turns) uses roughly UEC 0.6–0.85, EMD 0.3–0.6, STCI 0.4–0.7, CDL 0.3–0.6, ARR 0.7–1.0, while Short Shock (8–16 turns) uses UEC 0.5–0.75, EMD 0.2–0.4, STCI 0.6–0.9, CDL 0.5–0.8, ARR 0.4–0.7. "Apprehension" is then simply the same curve shape with lower bands: UEC around 0.4–0.6, EMD 0.1–0.3, STCI 0.3–0.5, CDL capped around 0.4–0.5, ARR still high so ambiguity remains but is carried by a gentler ride.[45]

Playtesting is explicitly telemetry-driven: every session route envelope logs per-turn metrics and at the end computes derived measures like average CDL, time outside target bands, max DET overshoot, and ARR-band hit rate, plus a comfortOutcome enum (`withinBand`, `tooIntense`, `tooFlat`). You then correlate these aggregates with persona and runContract IDs to see whether target "Apprehension", "Dread", "Panic" bands actually feel that way in practice; when runs regularly land in `tooIntense` or `tooFlat`, you tighten bands or adjust curves. This framework also supports AB tests: small strategy variants that alter "rise per turn" inside the same bands, tagged with `variantId` in telemetry, so you can compare pacing styles without ever leaving the declared intensity tier.[45]

47. How `metric_bands` drive H.Director

`metric_bands` are implemented as **soft targets with hard clamps**, not pure ceilings or pure probabilities. Each director persona and runContract declares target bands for UEC, EMD, STCI, CDL, ARR; per turn, the persona computes an "intended" movement (e.g., nudge UEC up, drop CDL) and passes it through `H.Director.constrainMetrics`, which clamps the request into the intersection of persona, runContract, consent, and comfort policies.[45]

Inside the band, they function as weighted preferences and gentle steering: selectors and node scoring compute a base score from invariants, then add a small "metric steering" term that favors candidates whose expected deltas move the session closer to band centers, with low amplitude so history and geography still dominate. At the edges, they become de-facto hard limits: comfort policies add per-turn and per-window delta caps (e.g., `maxDeltaUECPerTurn`), and director logic is forced into cooldown or "epilogue" modes when, for example, DET/CDL or ARR would otherwise exceed allowed spans. So the design answer is: they are soft steering and probabilistic bias *inside* their range, enforced as hard clamps at the boundary.[45]

48. Branching paths inside one `experience_contract`

Yes: a single `experience_contract` is explicitly defined as a preset profile that can bind multiple runContracts, director personas, and branches, all under its metric/invariant envelope. The spec suggests "Guided Ritual Investigation" and "Process-Driven Catastrophe Run" as examples: each experienceContract declares metric

bands (UEC, EMD, CDL, ARR), invariant envelopes (CIC, AOS, DET, SHCI ranges), and a reference persona, but those can fan out into different dungeonRunContracts and node graphs.[45]

Branches then become run-local differences in structure and local bands, not different global contracts: one branch can aim at higher CDL and lower ARR (more closure), another at lower CDL and higher ARR (haunting ambiguity), but both must lie within the experienceContract's overall band ranges. Director personas and H.Director logic track which branch the player is currently on and apply the same constrain-then-steer algorithm, so branches feel like different *flavors* of the same preset rather than totally separate games.[45]

49. Using Comfort_Policies for pacing (Safe vs Tense bands)

Comfort policies are the pacing governor that sit alongside metric bands and invariants. A director comfort policy defines things like `maxDetPerSession`, `requiredCooldownRatio` (minimum low-DET time per high-DET time), and an allowed CDL span; H.Director and H.Run must respect these when planning and executing beats.[45]

A "Safe_Band" is simply the policy's cooldown regime: low DET band, lower CDL, with selectors required to pick safe rooms, hubs, or low-intensity moods until quotas are met; within this band, exploration is allowed and branch density can be higher because stakes are low. A "Tense_Band" is a policy state where requiredCooldownRatio has been satisfied and the director is allowed to schedule higher-DET, higher-CDL beats; here you can enforce more linear progression by downranking optional loops and liminal detours until the current tension arc resolves. Runtime, H.Director can flip between these modes based on DET/CDL trajectories: when cooldown ratio dips, force Safe_Band; when satisfied and metrics are centered, allow Tense_Band and more constrained routing.[45]

50. Defining and authoring a new `runContract`

The process for a new `runContract` is already outlined as a research object: it becomes a Phase-2 bundle type with its own schema and explainer under HorrorPlace-Constellation-Contracts. The schema (`schemas/runtime/run-contract.v1.json`) defines fields like `runId`, `eventSequence` (ordered eventContract IDs), `moodCurves` (moodContract IDs plus stage curve), `difficultyProfile` (novice/standard/expert with UEC/EMD/STCI/CDL/ARR bands and persona skill expectations), invariant bands (CIC, AOS, DET, SHCI), and a `deathRiskPolicy` for how near-failure branches are narrated (always implication-only).[45]

The explainer doc (`docs/runtime/run-contract-explainer-v1.md`) then gives designers numeric examples—e.g., novice vs expert bands and which archetypes they map to—and describes how CHATDIRECTOR validates runs by aggregating per-event deltas to ensure the run's target bands are satisfiable without violating tileClass or stage bands. Authoring tools (and CI) enforce adherence: manifests route runContracts to the correct tier; `validateinvariants` checks their invariant/metric consistency; and AI-Chat authoring contracts insist that any planned run references a valid runContract ID, so no free-form script can bypass the spine.[45]

⁂

| 51 | `[DESIGN]` How are `preset_modes` (e.g., 'Survival', 'Exploration', 'Psychological') intended to modify the baseline behavior of the `H.Director` and the interpretation of metrics? |

52	`[DESIGN]` How should prompt templates be structured to effectively guide the AI-driven `H.Director`'s persona? What fields should they contain to specify tone, pacing, and thematic direction?
53	`[DESIGN]` What are the key levers for tuning a `DirectorPersona`? Is it a continuous spectrum or a set of discrete personalities (e.g., 'Whisperer', 'Grandmaster', 'Chronicler')?
54	`[DESIGN]` How can designers ensure that encounters defined in `Atrocity-Seeds` are both novel and consistent with the overall tone dictated by the `DirectorPersona` and current `metric_bands`?
55	`[DESIGN]` What is the relationship between a `replayable_run` and the `History` and `Dead-Ledger`? How is the state saved and restored to guarantee identical outcomes upon replay?

Preset modes, director personas, atrocity seeds, and replayable runs are all thin contract layers over the same invariants and metric spine; they never bypass `H.Director`, `H.Selector`, or Dead-Ledger, they just change how those systems steer and interpret UEC, EMD, STCI, CDL, ARR.[47] [48]

51. Preset modes and H.Director

`preset_modes` like "Survival", "Exploration", and "Psychological" are intended as user-facing shorthands for different combinations of experienceContracts, directorPersonas, and comfort policies, not as separate engines. Under the hood, each mode is a small profile (an AI-Chat horror profile / experienceContract) that remaps the same metrics into different priorities and trade-offs: for example, a Survival preset emphasizes DET and STCI, clamps CDL tightly, and uses ARR to ensure some closure, while an Exploration preset raises EMD and ARR but keeps DET lower and allows more branch density.[48] [47]

For `H.Director`, the mode changes three things: which directorPersona is bound (e.g., Cold Overseer vs Silent ArchivistDirector), what metric bands it targets for UEC/EMD/STCI/CDL/ARR, and which comfortPolicy it references (different DET budgets, cooldown ratios, CDL spans). Interpretation of metrics also shifts: in a Psychological preset, high CDL and mid-high ARR are "on purpose" and tolerated within policy; in a Survival preset, the same CDL might be treated as a warning of confusion and trigger more explanation, while STCI and DET peaks are allowed to be sharper.[47] [48]

52. Prompt templates for H.Director personas

Prompt templates are structured contracts, not ad-hoc prose, and they need enough fields for the runtime to map persona, strategy, and metrics into concrete outputs. The design work around director strategies ("clinical recounting", "ritual invocation", "archival detachment", "gentle guide", "ominous implication") already assumes templates carry a tone label and metric hints the Director can use.[47]

A good template schema for AI-driven `H.Director` includes: a `templateId` and `directorKind` (Cold Overseer, Ritual Statistician, Silent ArchivistDirector, etc.); a `toneLabel` (clinical, ritual, archivist, gentle, ominous), and a `strategyTag` (e.g., narrative, hint, recap, cooldown). It should also declare a `metricEnvelope` (intended bands or deltas for UEC, EMD, STCI, CDL, ARR), `stage` (buildup, peak, cooldown, epilogue), and flags for `explicitnessLevel` and valid invariant regimes (e.g., which DET/SHCI/LSG bands it assumes), so `H.Director` and `H.Safety` can filter them before use. Finally, templates for safety outcomes need `safetyStrategyTag` (soften, imply, downshift) so `H.Director.applySafetyDecision` can swap in persona-correct language when guardrails fire.[48] [47]

53. Tuning DirectorPersona: levers and spectrum

The stack treats a DirectorPersona as a contract object with both continuous levers and discrete identities. Each persona declares invariant bands (CIC, AOS, DET, SHCI) and metricTargets (UEC, EMD, STCI, CDL, ARR) plus a comfortPolicy (max DET per session, requiredCooldownRatio, allowedCDL span), which are all continuous ranges the runtime clamps against. On top of that, persona skill profiles (analysisSkill, composure, worldviewRigidity) modulate how strongly clues raise UEC/EMD, how much DET/CDL each threat event contributes, and how quickly ambiguity collapses into resolution.[48] [47]

At the same time, there is a small, named catalog of director "skins" such as Cold Overseer, Ritual Statistician, Silent ArchivistDirector, with associated strategies like clinical recounting, ritual invocation, archival detachment, gentle guide, and ominous implication. So in practice you get discrete persona types (e.g., "Whisperer" or "Chronicler" would be new directorKinds) defined as points in a continuous parameter space: each identity is a fixed bundle of metric bands, invariant envelopes, strategy mix, and tone, which you can tune numerically without proliferating infinite personalities.[47] [48]

54. Atrocity-Seeds, novelty, and consistency

Atrocity-Seeds are region/seed-level contracts that define haunt logic and trauma echoes; they sit in their own repo but obey the same invariant and metric constraints. Novelty comes from the seed's internal pattern graph, unique invariant signatures (CIC/SHCI/AOS/LSG/HVF combinations), and event/mood bundles, while consistency comes from binding selection to current directorPersona, experienceContract, and metric_bands.[48] [47]

Designers ensure consistency by: (1) giving each seed explicit invariant envelopes and per-event metric deltas, so `H.Selector` and `H.Director` can only pick encounters whose expected effects fit the current effectiveMetrics; and (2) tagging seed events with strategy and tone hints compatible with persona strategies (e.g., clinical vs ritual vs archivist), so the same underlying atrocity is narrated differently but still inside persona bands. Novelty is preserved by selector history: route telemetry tracks which patterns from a seed the player has seen; `H.Selector` avoids repeating the same high-impact events within a session and varies which branches or manifestations are

surfaced across runs, while invariants keep the seed's tone and themes aligned with its region history and the director's envelope.[47] [48]

55. Replayable runs, History, and Dead-Ledger

A `replayable_run` is defined as a runContract + experienceContract + directorPersona combination whose behavior is fully determined by contracts, invariants, and a seedable selection process, not by ad-hoc randomness. The History layer provides immutable region/seed contracts and invariant fields (CIC, SHCI, AOS, etc.), while Dead-Ledger provides attestations and decision tokens (deadledgerref) that prove which bundles/personas/seeds were authorized for that run.[48] [47]

To guarantee identical outcomes on replay, the system stores a compact "route envelope" for the session that includes: runContractId, region/seed IDs, directorPersonaId, experienceContractId, the RNG seed or selector variantId, and the per-turn sequence of selectorPatternIds, nodeIds, and safety/director strategies applied. Because all content selection goes through `H.Selector`, `H.Director`, and Policy.DeadLedger with deterministic rules and the same seed, re-running with the same contracts, invariants, seed, and consent state yields the same sequence of patterns, safety decisions, and epilogue, as long as Dead-Ledger returns equivalent allow/allow-with-caps tokens for the referenced bundles.[47] [48]

⁂

| 56 | `[DESIGN]` How is the `death-risk_library` structured? What metadata defines the risk profile of an encounter, and how is this profile interpreted by the `H.Safety` module? |

57	`[DESIGN]` How can designers use the `LSG` (Liminal-Space-Guardrails) to create procedurally generated liminal environments that feel coherent and unnerving?
58	`[DESIGN]` What guidance should be provided to authors of `Atrocity-Seeds` regarding the use of the `CIC` and `AOS` invariants within their content?
59	`[DESIGN]` How can the system support the creation of non-linear narratives where the `H.Selector`'s choices have significant, lasting consequences on the `History`?
60	`[DESIGN]` What is the best practice for designing `experience_contracts` that are modular and can be combined with different `Atrocity-Seeds`?

`death-risk_library` entries, liminal-space guardrails, atrocity authorship, non-linear history, and modular experience contracts all sit on the same spine: invariants (CIC, AOS, LSG, SHCI, etc.), entertainment metrics (UEC, EMD, STCI, CDL, ARR), and run/experienceContracts that `H.Selector`, `H.Safety`, and `H.Director` treat as authoritative.[49]

56. Death-risk library and H.Safety

The `death-risk_library` should be a policy-level catalog that describes how "risk of death" is *routed* and narrated, never depicted, keyed per runContract difficulty and encounter type. Structurally, each entry is a small policy object that lives alongside runContracts and mechanicContracts, carrying fields like `riskTier` (none, low, medium, high), `outcomeMode` (gentle-drift, hard-branch, near-failure), allowed DET deltas, CDL and ARR targets, and `allowedSurfaces` (implication-only, offscreen).[49]

From the H.Safety side, this risk profile is interpreted as *constraints* and *fallback strategies* rather than permission to escalate. The pipeline already defines ABSOLUTE/CONSENTGATED/BUDGETGATED invariants and a five-phase evaluation order; the death-risk profile tells H.Safety which branch style to choose when death-like outcomes are in play, so violations trigger imply/downshift into an offscreen, short epilogue, not graphic content. For a given turn, `H.Safety.evaluateTurn` and `H.Director.applySafetyDecision` use the encounter's death-risk metadata plus current budget snapshot and consent caps to select strategy verbs like `imply` or `downshift`, then templates implement "you are gone / the route closes" as quiet branch termination.[49]

57. Using LSG for liminal environments

LSG and HVF turn liminality into a numeric field over the map, which you then let selectors and node contracts exploit. Nodes tagged with liminal roles (doorways, stairs, thresholds, tunnel mouths) should carry high LSG (for example 0.6–1.0) and moderate-to-high HVF magnitude, marking them as preferred sites for transitions, glitches, and spectral echoes.[49]

Procedural generation can then treat the node graph as a “dread field”: sample LSG and SHCI across tiles, cluster high-LSG corridors as ridges where crossings feel stressful, and keep low-LSG, low-HVF regions as neutral connective tissue. `H.Selector` uses these bands when picking patterns, so high-LSG nodes preferentially host glitch/echo events and subtle manifestations tied to SHCI/SPR, while DET remains controlled by comfort policy and tier; coherent unnerving spaces arise because the same LSG/HVF gradients drive both layout and encounter placement.[49]

58. Guidance for Atrocity-Seed authors on CIC and AOS

Atrocity-Seeds are history bundles; authors should treat CIC and AOS as the primary “physics” knobs rather than flavor. CIC encodes how strong and localized the catastrophic imprint is, while AOS encodes how opaque or contradictory the records are; together they determine whether a seed feels like a deep, static haunt (high SHCI, high CIC, modest AOS) or a restless, shallow haunt (high AOS, moderate SHCI).[49]

Guidance for authors: for anchor events and hubs, aim for high CIC (0.6–0.8) with mid AOS (0.2–0.5) so the trauma is heavy but legible, which supports resolution beats and lower ARR at the end of a run. For liminal or side-path atrocities, push AOS high (0.6–1.0) and keep SHCI moderate so the seed yields many conflicting fragments and rumors, raising EMD and ARR without ever fully resolving; selectors then use those envelopes plus SHCI/SPR policy to classify manifestations as glitch/echo/apparition in a consistent way.[49]

59. Non-linear narratives and lasting History consequences

The stack already assumes `H.Selector` operates over history-selector rules and patterns with Dead-Ledger-backed bundles and a session route envelope; making choices “stick” in History is about which contracts you mutate and which you merely reference. Non-linear runs should treat major forks as decisions that emit selector-decision and mechanic events with small, declarative outcome vectors over UEC, DET, CDL, ARR plus semantic tags (brave, cautious, avoidant, curious), all recorded in the route envelope and telemetry.[49]

To give those decisions lasting consequences, define higher-level History mutation contracts (for example “mark region seed as cooled” or “unlock new director behaviors”) that are only authorized when specific selectorDecision patterns appear, and persist those as new registry entries or flags referenced by future runs. Dead-Ledger then attests these state changes; subsequent sessions querying the same region see different invariant envelopes or available seeds, but still via `H.Selector` and Policy.DeadLedger tokens, so non-linearity remains governed and replayable rather than arbitrary branching.[49]

60. Designing modular, combinable experience_contracts

Experience contracts are thin presets that bundle metric bands, invariant envelopes, director persona, and difficulty into named modes, leaving actual horror content to Atrocity-Seeds and runContracts. Best practice is to keep them seed-agnostic: each experienceContract declares targeted bands for UEC, EMD, STCI, CDL, ARR; bounded ranges for key invariants (CIC, AOS, DET, SHCI); and references a directorPersona whose metricTargets lie inside those bands, but it does not bake in specific events.[49]

Compatibility with different Atrocity-Seeds then becomes a matter of overlap: `H.Selector` only pairs an experienceContract with seeds whose invariant envelopes (CIC/AOS/SHCI/LSG/HVF) and internal metric deltas can be clamped into that experience’s ranges. Designers should therefore keep experienceContracts relatively broad (for example Slowburn Dread vs Short Shock vs Guided Ritual Investigation) and encode stricter, seed-specific behavior in runContracts and seed metadata; this lets the same contract drive multiple regions while the spine (invariants + metrics + comfort policy) ensures consistent tone and pacing.[49]

⁂

| 61 | `[DESIGN]` How can designers use the `SHCI` (Spectral-History-Corruption-Invariant) to make the game world react to past events in a meaningful way? |

| 62 | `[DESIGN]` What is the process for balancing the difficulty of encounters across different `metric_bands` to ensure a smooth escalation of tension? |
| 63 | `[DESIGN]` How can the `image-helper` behavior be parameterized by a `runContract` to match the desired aesthetic and horror theme of a specific session? |
| 64 | `[DESIGN]` What is the workflow for playtesting a new `experience_contract` to validate its pacing and horror

effectiveness against the defined `metric_bands`? |
| 65 | `[DESIGN]` How should the `Orchestrator` handle conflicts between different `invariant_requirements` specified in a `runContract` and those of a selected `Atrocity-Seed`? |

SHCI, metric-band difficulty, image-helper, playtest workflow, and invariant conflict handling all plug into the same spine: invariants (CIC, AOS, SHCI, LSG, etc.), entertainment metrics (UEC, EMD, STCI, CDL, ARR), and contracts that CHATDIRECTOR / Orchestrator validate and route.[51]

61. Using SHCI so the world reacts to past events

Designers should treat **SHCI** as "how tightly local phenomena are chained to documented history and prior decisions," not as a generic "spooky level." Region and seed contracts already expose SHCI bands; overseer-style personas are only allowed where SHCI and CIC are high, which means manifestations there should be strongly history-locked, not random.[51]

To make the world react meaningfully, use SHCI in two layers. First, in selection: `H.Selector` should prioritize patterns whose `historyCoupling` matches local SHCI, and give extra score to patterns that reference events already logged in this player's cross-session state (completed lore threads, past failures), so echoes literally reuse their own route history. Second, in mutation: define small History-mutation contracts that, when certain selector decisions fire in high-SHCI regions, flip flags in the region's "current state" (for example "gate sealed", "catacomb cooled"); future runs read those flags as modifiers on SHCI or on seed availability, so the same place gradually shifts from violent apparitions (high SHCI, high DET) to distant archival murmurs (still high SHCI, lower DET, higher ARR).[51]

62. Balancing encounter difficulty across metric_bands

Difficulty is expressed numerically via **metric_bands** in runContracts and experienceContracts, plus skill/budget policies; encounter difficulty is "how much a beat moves you through those bands." A smooth escalation curve means mapping difficulty tiers (Novice / Standard / Expert) to explicit deltas in UEC, EMD, STCI, CDL, ARR and DET classifications (low / medium / high), then ensuring each beat's expected deltas keep the trajectory inside the contract's envelope.[51]

The process is: start from the runContract's target bands (for example Slowburn vs Short Shock templates), and classify each encounter as mainly UEC-raising, EMD-raising, STCI spike, CDL spike, or ARR-collapsing. For each difficulty, define allowed per-beat ranges (for example, Novice encounters may raise DET only within lowDetBand, CDL in 0.1–0.3, while Expert can use medium/high bands), and have `H.Selector` score out patterns whose deltas would overshoot when combined with recent beats, using the shared DET band classifier and budget state from H.Budget. Telemetry (route envelopes + derived metrics like maxDetOvershoot, avgUecDeviation) then shows whether escalation is too abrupt; designers adjust per-encounter deltas or runContract bands and replay until curves stay within target.[51]

63. Parameterizing image-helper by runContract

The **image-helper** is already envisioned as `H.ImageHelper.describe(context, invariants, metrics, inventory)`, returning an implication-safe description, hidden clue tags, and suggested metric deltas. To make it session-specific, let the **runContract** supply an `imageStyleProfile` and difficulty mapping: aesthetic tags (industrial fog, ritual chalk, archival grain), allowable explicitness (always implication, no bodies, etc.), and minimum symbol clarity per difficulty.[51]

At runtime, CHATDIRECTOR passes the active runContractId and difficulty into the context, and `H.ImageHelper.describe` reads the runContract's `imageStyleProfile` to choose motif palettes and density of hints. For example, a Guided Ritual Investigation contract might enforce one clear symbolic clue per image at Novice, with CDL kept low and ARR high, while a Process-Driven Catastrophe Run allows more abstract symbols and higher CDL for Expert, but still clamps DET and explicitness via the same consent/budget caps. Because image-helper only ever emits text-level descriptions plus `hiddenTags` and suggested metric deltas, it remains Tier-1 safe while still aligning tightly with each session's aesthetic and horror theme.[51]

64. Playtesting an experience_contract against metric_bands

Playtesting a new **experience_contract** is about validating that actual metric trajectories match the contract's target bands and that comfort outcomes are acceptable. The system already defines session metrics envelopes and route envelopes with UEC, EMD, STCI, CDL, ARR curves and a derived `comfortOutcome` field (withinBand, tooIntense, tooFlat).[51]

The workflow is: implement the experienceContract as a JSON preset (metric bands, invariant envelope, directorPersona, difficulty range), ensure CHATDIRECTOR validates it against tier caps, then run controlled sessions across tiers and player profiles. Each session emits a route envelope and session-metrics envelope; offline analysis compares observed trajectories to targets (for example, average CDL vs band, time spent outside bands, ARR at ending) and counts safety downshifts from H.Safety/H.Director. Designers then adjust the contract (tighten bands, tweak director comfortPolicy, or alter which seeds are eligible) until most runs land on `comfortOutcome = withinBand` with the intended DET and CDL peaks, and replayability is acceptable (e.g., diversity metrics from H.Selector's history state indicate multiple distinct routes).[51]

65. Orchestrator handling invariant_requirements conflicts

The **Orchestrator** sits above contracts and must treat invariant_requirements as hard constraints: runContracts and Atrocity-Seeds both declare invariant envelopes (bands for CIC, AOS, DET, SHCI, LSG, etc.), and only compatible pairs should ever be bound. When a runContract's `invariant_requirements` clash with a candidate Atrocity-Seed's region/seed invariants, Orchestrator should detect this at planning time via the history-selector-rule and conflict-policy schemas, not at runtime.[51]

Best practice is: define a `history-selector-conflict-policy` that codifies merge rules (for example, intersect bands and see if the intersection is non-empty, or treat certain fields as non-negotiable, such as DET caps from consent tiers vs SHCI minima for overseer personas). At orchestration time, CHATDIRECTOR/Orchestrator attempts to bind runContract + experienceContract + Atrocity-Seed; if the intersection of invariant envelopes is empty or violates SAF/consent tier ceilings, it either selects a different seed or downshifts to a compatible experienceContract, and logs a conflict event. CI and manifest validation should also prevent publishing combinations whose required invariants can never be satisfied, by running the same intersection logic against contract cards in HorrorPlace-Constellation-Contracts.[51]

⁂

| 66 | `[DESIGN]` What documentation should be created for `H.Selector` patterns to help designers understand the available algorithms for node selection (e.g., random, greedy, pathfinding-based)? |

67	`[DESIGN]` How can designers create encounters that are sensitive to the player's previous choices, as recorded in the `History`?
68	`[DESIGN]` What is the recommended approach for creating a 'safe mode' or 'casual mode' that adjusts the `metric_bands` and `comfort_policies` for less experienced players?
69	`[DESIGN]` How can the `CIC` be designed to be transparent to the player, providing a sense of agency without breaking immersion?
70	`[DESIGN]` What criteria should be used to evaluate the success of an `experience_contract`? Should it be based on quantitative metrics (ARR, UEC) or qualitative feedback?

Designers should treat H.Selector, invariants, and experienceContracts as the shared "physics," then layer safe-mode presets and evaluation criteria on top; none of these answers add new surfaces, they tie into the existing selector rules, metric spine, and CHATDIRECTOR governance.[52]

66. Documentation for H.Selector patterns and algorithms

The selector already has a formal rule schema (`history-selector-rule-v1.json`) and pattern schema planned; documentation should mirror that contract structure rather than freeform prose. You want a short spec in HorrorPlace-Constellation-Contracts (for example `docs/runtime/history-selector-lua-api-v1.md`) that explains how `H.Selector.selectPattern` (or `choosePattern`) consumes the rule catalog: nine phases from `safety-prefilter` through `final-rank`, each with actions like `EXCLUDE`, `SCORE-MODIFY`, and a clamped scoring model.[52]

On top of that, add a pattern authoring guide (for example `docs/authoring/history-selector-patterns.md`) that: enumerates supported algorithmic styles (pure random within safe set, greedy on score, diversity-biased, pathfinding over node graphs) and shows how to encode them using rule parameters and pattern metadata (tags, invariant envelopes, cooldowns). For designers, include concrete examples of "random but gated by safety," "escalation curve using lastIntensity and sessionDepth," and "pathfinding-style selection where patterns embed graph positions and the selector's scoring kernel simulates a walk," all referencing the same sessionStateContract and crossSessionStateContract fields.[52]

67. Encounters sensitive to player History

History sensitivity is already baked into the selector rule schema via `crossSessionStateContract` fields like encounter history and lore thread progress; designers need to use those fields instead of reinventing state. Each encounter (pattern or node) should carry prerequisites and payoff tags tied to this History view: required or forbidden pattern IDs, minimum or maximum counts of specific event types, and thresholds on "thread completion" metrics.[52]

In practice, you define patterns like "echo of a past failure" whose score boosts only if the corresponding earlier pattern appears in `servedContentIds` for the session, or even in prior sessions' cross-session history. Rules such as SEL-R007 (diversity bonus) and SEL-R004 (escalation curve) can also look at History to penalize repeating the same pattern families too often and encourage revealing new facets of an atrocity the player has partially uncovered. Designers then think in terms of "if the player spared X earlier, favor patterns that show the consequences" by encoding that relationship in pattern metadata and letting H.Selector's rule engine handle the scoring.[52]

68. Designing safe / casual modes via metric_bands and comfort_policies

A safe or casual mode is just an experienceContract plus director comfortPolicy with very conservative bands, validated against the same consent and budget tiers. The recommended approach is to add one or more "casual" experienceContracts that set lower DET caps, narrower CDL spans, and higher ARR and UEC bands (more mystery, more resolution, less dissonance), and to bind them to gentler personas and budget policies.[52]

Concretely, a "Casual Haunt" preset might specify UEC 0.4–0.7, EMD 0.1–0.3, STCI 0.3–0.6, CDL 0.1–0.4, ARR 0.7–1.0, with comfort policies that limit high-DET events per session and enforce high cooldown ratios. H.Safety and H.Budget already clamp intensity based on consent tier and budget; safe mode simply narrows those caps further and biases selectors toward low-DET moods and implied events, while telemetry and the comfortOutcome field ensure you can see if casual runs are still "too intense" or "too flat" and adjust bands accordingly.[52]

69. Making CIC transparent without breaking immersion

CIC is already the "Catastrophic Imprint Coefficient" behind the scenes; to players it should feel like "how scarred a place is" and how much their choices matter there. The transparent design is to expose CIC indirectly as in-world affordances: regions with higher CIC offer richer lore, more consequential branches, and denser clues, while low-CIC regions feel like transient or safe zones.[52]

You can surface this through UI hints ("This route is deeply marked by past disasters; choices may have lasting consequences") tied directly to CIC bands in the region/seed contracts, and through mechanical behavior: H.Selector and H.Director treat high-CIC regions as eligible for overseer personas and more permanent History mutations, while low-CIC areas rarely change the world. Because these explanations and behaviors map 1-to-1 to CIC values but are framed diegetically (omens, archivist commentary, difficulty labels), players feel agency and warning without seeing a raw numeric stat.[52]

70. Evaluating experience_contract success

ExperienceContracts are governance artifacts, so they should be evaluated both quantitatively (against the metric spine) and qualitatively (player comfort and enjoyment). Quantitatively, route envelopes and session-metrics envelopes provide UEC, EMD, STCI, CDL, ARR curves and a derived `comfortOutcome` plus secondary metrics like `maxDetOvershoot`, `avgUecDeviation`, and counts of safety downshifts.[52]

A contract is "working" if, across many sessions, its observed trajectories stay inside or near its bands (small deviation, low overshoot), comfortOutcome is usually `withinBand`, and replayability metrics indicate enough route diversity. Qualitatively, you layer in structured feedback: post-run surveys or implicit signals (opt-outs, replays, mode preferences) mapped back to the same contractId, then look for correlations between CDL/ARR profiles

and reported satisfaction. The recommended practice is to treat quantitative metrics as the hard gate (no experienceContract that regularly blows DET/CDL caps or produces frequent tooIntense outcomes stays in public tiers), and use qualitative feedback to tune within the safe envelope toward “fun, memorable, and on-brand” horror.[52]

⁂

| 71 | `[DESIGN]` How can the `H.Director` be programmed to introduce moments of false safety or lull the player into a false sense of security? |

72	`[DESIGN]` What is the best way to calibrate the `STCI` to reflect genuine spectral tension rather than just random event triggers?
73	`[DESIGN]` How can designers use the concept of 'liminal routing' to create mazes or pathways that feel disorienting and impossible?
74	`[DESIGN]` What guidelines exist for creating `Atrocity-Seeds` that are truly reusable across different contexts and `runContracts`?
75	`[DESIGN]` How should the `H.Director` balance scripted sequences with emergent events driven by the `H.Selector`'s choices?

H.Director, STCI, liminal routing, reusable Atrocity-Seeds, and the balance between scripted and emergent content all sit on the same governed spine: selector patterns, invariants (CIC, LSG, SHCI, etc.), entertainment metrics (UEC, STCI, CDL, ARR), and contracts routed by CHATDIRECTOR.[53]

71. H.Director and false safety

H.Director should introduce **false safety** by treating lull phases as first-class segments in the runContract and selector patterns, not as ad-hoc “gotcha” jumps. In practice, escalation patterns like “Classic Slow Burn” already define beats and cooldown fillers such as `PAT-COOL-001 Safe Harbor`; you assign those beats low DET and moderate STCI, then let H.Selector and H.Budget enforce cooldown ratios before new high-DET events are allowed.[53]

H.Director’s persona and comfortPolicy can then explicitly target cycles where STCI is dialed down and ARR remains high during lull beats, while the escalation curve rule (SEL-R004) ensures tension eventually rises again as budget and session depth increase. To the player, this feels like genuine safety because multiple systems agree: selector chooses Safe Harbor-like patterns, templates soften tone, image-helper reduces CDL contributions, and H.Safety keeps jumpscares gated by consent and budget; then, once metrics and cooldown rules permit, the next escalation arc resumes.[53]

72. Calibrating STCI for real spectral tension

STCI (Safe-Threat Contrast Index) should reflect how clearly the difference between “safe” and “unsafe” states is communicated over time, not simply whether an event is spooky. Calibration starts at contract level: runContracts and experienceContracts define STCI bands (for example, Slowburn vs Short Shock profiles), and selector patterns encode where contrast should spike (ESCALATIONARC beats) versus where it should flatten (COOLDOWNFILLER).[53]

In the scoring engine, rules like escalationcurve (SEL-R004) and patternmatch (SEL-R006) work together so that high-STCI patterns are picked when the run is supposed to crest, but only if budget and consent allow it; low-STCI events are favored when cooldown or safety fallbacks are active. Telemetry via session-metrics envelopes then measures STCI alongside DET and CDL; if you see random, noisy swings instead of deliberate ramps (for example, high STCI spikes during nominal cooldown), you tighten rule parameters or adjust pattern STCI deltas until trajectories line up with the intended arcs.[53]

73. Liminal routing and impossible mazes

“Liminal routing” is the idea that routing itself respects LSG (Liminal-Space gradient) and HVF (Haunt Vector Field) so spaces feel disorienting but still grounded in invariants. Regions and tiles have LSG and HVF bands; high-LSG, high-HVF edges should host thresholds, loops, and ambiguous transitions, while low-LSG areas remain relatively stable.[53]

Selector patterns and PCG logic can then treat the node graph as a vector field: routes along high-LSG ridges are allowed to "fold" back on themselves (for example, corridors that visually repeat, doors that return to earlier nodes) as long as the underlying selector state and History remain consistent. Patterns tagged as liminal or maze segments carry specific LSG/LSG-delta requirements; SEL-R008 (regional affinity) and diversity rules prevent overuse, while telemetry and runContracts ensure DET/CDL stay within bands so "impossible" spaces feel uncanny rather than frustrating.[53]

74. Guidelines for reusable Atrocity-Seeds

Reusable Atrocity-Seeds should be designed as **history bundles + invariant envelopes**, not scenario scripts, so they plug cleanly into multiple runContracts. Each seed defines: its CIC/AOS/SHCI/LSG/HVF bands; a catalog of events and moods expressed as patterns with metric deltas; and clear tags for region types, tiers, and director personas that are allowed to use it.[53]

To keep seeds truly reusable, avoid hardcoding pacing or difficulty; instead, annotate each event with relative impacts on UEC, STCI, CDL, ARR and let runContracts clamp or scale those effects to fit their metric bands. History-selector rules then decide which slice of the seed manifests based on context: in a Slowburn run, only low-DET, high-ARR fragments may appear, while in a Process-Driven Catastrophe run, the same seed can surface more aggressive, process-heavy branches—but always via the same invariant envelope and safety/budget caps.[53]

75. Balancing scripted sequences and emergent selector events

Scripted sequences should be modeled as **selector patterns and runContract bindings**, not special-case engine logic, so they coexist with emergent events in the same scoring and safety pipeline. The pattern library already includes types like ESCALATIONARC and REGIONALNARRATIVE, which can encode multi-beat scripted arcs (for example, "Subterranean Descent") while still going through SEL-R001–SEL-R009 scoring and SAF/consent/budget gates.[53]

H.Director's role is to choose how strongly to bias toward these scripted arcs versus opportunistic patterns, based on persona, stage, and metrics: early in a run, it may require that an ESCALATIONARC pattern remains active, while later it can let H.Selector's diversity and budget rules favor emergent, history-sensitive beats. The best practice is to reserve a fraction of pattern budget per stage for "must-hit" scripted beats (for narrative coherence) and leave the rest to selector-driven emergent events, with H.Director only influencing weights and safety strategies, never bypassing the shared selector and safety machinery.[53]

⁂

| 76 | `[DESIGN]` What is the process for creating a library of `director_personas` that are distinct, memorable, and offer unique gameplay challenges? |

77	`[DESIGN]` How can the `SHCI` be used to create a persistent, evolving threat that corrupts the environment over the course of a session?
78	`[DESIGN]` What is the recommended method for testing the robustness of a `runContract` against unexpected player actions that fall outside the intended flow?
79	`[DESIGN]` How can designers leverage the `Dead-Ledger` to create a meta-narrative about past failures or atrocities within the game world?
80	`[DESIGN]` What is the standard procedure for versioning `experience_contracts` and `Atrocity-Seeds` to ensure long-term compatibility and modularity?

Director personas, SHCI-driven corruption, robust runContracts, Dead-Ledger meta-narrative, and versioning all plug into the same doctrine: contracts are Phase-2, tier-aware objects on the CIC/SHCI/DET spine, with telemetry and Dead-Ledger giving you long-term memory and governance.[54]

76. Building a library of director_personas

The process for a **director_persona** library should be contract-first, not prose-first. You already have a pattern for director-style roles (Cold Overseer, Ritual Statistician, Silent ArchivistDirector) as numeric envelopes over CIC, AOS, DET, SHCI and UEC, EMD, STCI, CDL, ARR, plus tone labels and comfort policies. The first step is to formalize `directorPersonaContract` as a Phase-2, Tier-aware objectKind with fields for `directorKind`, `toneLabel`,

`allowedTier`, `invariantBands`, `metricTargets`, and `comfortPolicyRef`, so every persona is just data that CHATDIRECTOR can validate.[54]

From there, you design a small but diverse set of archetypes by varying three axes: emotional style (clinical, ritual, archivist), pacing profile (how aggressively they move STCI and DET), and interaction rules (how they use gifts, hints, and punishments). Each archetype then gets 1–2 concrete persona instances with slightly different bands and comfort policies, creating distinct gameplay challenges: a Cold Overseer who demands calculated risk under high SHCI, a Ritual Statistician who pushes high CDL/EMD puzzles, a Silent Archivist who trades higher UEC for gentler DET but relentless SHCI. Telemetry (sessions per directorKind, metric drift, safety downshift count) and experienceContracts then tell you which personas feel memorable and fair; you iterate by tuning numbers, not rewriting persona logic.[54]

77. Using SHCI as a persistent, evolving threat

SHCI is your **Spectral-History Coupling Index**; to make it feel like a persistent corruption, treat it as a slowly changing field that both constrains manifestations and mutates the environment as the run progresses. At runContract level, you already target SHCI bands; the next step is to let History and selector decisions feed back into a per-region SHCI "drift" value: completing certain patterns, triggering certain Atrocity-Seed beats, or failing key choices pushes SHCI up or down within the seed's envelope.[54]

Designers can then define pattern tags and environmental rules keyed to SHCI thresholds: at low SHCI only minor echoes appear and architecture is stable; at mid SHCI, geometry bends (liminal routing intensifies) and Atrocity-Seed motifs bleed into otherwise neutral nodes; at high SHCI, high-DET events unlock and even safe hubs begin to glitch. Because selectors, H.Director, and image-helper all read the same SHCI snapshot, the corruption feels coherent: lighting, prose, image motifs, and available choices all warp together, and when a session ends you can persist SHCI changes into cross-session History to make later runs inherit the scars.[54]

78. Testing runContract robustness against unexpected actions

A `runContract` is robust when unexpected player actions don't produce undefined behavior; they either route into safe lulls, fall back patterns, or governed refusals. Recommended testing is to treat "off-path" actions as a separate test suite: fuzz user intent and context labels at the H.Banter/H.Intent level (nonsense input, topic switches, refusal to engage with core puzzles), and check that selector + director + boundary always choose from valid patterns, apply safety decisions, and keep metrics within bands.[54]

On the contract side, you can define explicit **fallback patterns** (for example, generic COOLDOWNFILLER, neutral REGIONALNARRATIVE, or soft refusal templates) that are always eligible under the run's invariant/metric envelopes. CI and telemetry then help: a `run-session-metrics` envelope records completion, branch types, and where safety or selector denials occur; you flag any runContract where unexpected actions regularly cause hard denials, unbounded CDL spikes, or repeated Safe Harbor loops, then adjust pattern governance or metric bands until off-path behavior still feels diegetic and controlled.[54]

79. Using Dead-Ledger for meta-narrative

The Dead-Ledger is the system's high-trust record of serious violations, bans, and irreversible events; it's also a perfect spine for a **meta-narrative** about past failures and atrocities, as long as you never surface real-world details. At contract level, runContracts and directorPersonaContracts can carry `deadLedgerRef` fields that attest they were designed with specific governance incidents in mind (for example, "post-incident cooling profile"), without embedding any raw content.[54]

Designers can then create Atrocity-Seeds and regional lore that **echo** these recorded failures in abstract: a region whose CIC and AOS bands mirror those of a serious past incident; a Director who speaks of "prior sessions" that pushed too far and were sealed. In-world, H.Director and templates use this by hinting at nodes or routes "closed by decree," or by exposing rituals that symbolically repair or acknowledge the Dead-Ledger scars, while telemetry and governance ensure you never re-enact real incidents—only draw on their invariant patterns to shape fictional counterparts.[54]

80. Versioning experience_contracts and Atrocity-Seeds

ExperienceContracts and Atrocity-Seeds must be versioned like any other Phase-2 contract family: schema versions, object IDs, and compatibility rules in the spine. The standard pattern in HorrorPlace-Constellation-Contracts is to have each schema carry a `schemaRef` and semantic version (for example, `experience-contract.v1`, `atrocity-seed.v1`), while each instance uses a stable `id` plus optional `deadLedgerRef` and `descriptorId`, so CHATDIRECTOR and CI know exactly which shape they're validating.[54]

For **long-term compatibility**, changes that only tweak metric bands, comfort policies, or selector weights stay within the same schema and often the same contract `id` (recorded as new Git history with telemetry-backed justification). Structural changes—new fields, new invariant semantics, new pattern types—require a schema bump (v2, v3) and often a new contract `id`, with runContracts and directorPersonas explicitly referencing which version they expect, so selectors and H.Run can reject incompatible mixes at validation time. Atrocity-Seeds should be especially modular: keep the seed's historical/invariant envelope stable and evolve its internal pattern catalog over time, so older runContracts can still bind to a seed's v1 envelope even as newer runs take advantage of richer v2 events.[54]

⁂

Runtime Behavior, Nodes, and Encounters

This section presents approximately 30 questions concerning the real-time execution of the Horror.Place system. It focuses on the granular mechanics of how the `H.Director` and `H.Selector` operate during a live session, covering trigger logic, node selection, spectral entity behavior, and player-facing indicators of the system's state.

ID	Question / Query
81	`[RUNTIME]` What is the precise trigger mechanism for spawning a new encounter? Is it based on player location, timer, metric thresholds, or a combination of factors?
82	`[RUNTIME]` How does the `H.Selector` receive input to make a decision? Is it polled periodically by the `H.Director` or does it get notified of events via a pub/sub system?
83	`[RUNTIME]` What is the exact data payload passed from `H.Director` to `H.Selector` when requesting a node choice?
84	`[RUNTIME]` How does the `H.Selector` handle situations where no nodes from `Atrocity-Seeds` meet the current `metric_bands` and `invariant_requirements`? What is the fallback behavior?
85	`[RUNTIME]` What are the specific `selector_patterns` available for the `H.Selector` to choose nodes? (e.g., Random, Weighted by Metrics, Follow Predefined Path, Greedy Search).

Encounter spawning and node choice are already mostly defined by the H.Run loop, the history-selector rules, and the selector pattern catalog; below is how these five questions map onto that runtime behavior.[55]

81. Trigger mechanism for spawning encounters

A "new encounter" is spawned whenever **H.Run.nextturn** asks H.Selector for a pattern and then routes a template through H.Templates and H.Boundary; there is no separate hidden timer loop in the current design, it is input-driven. On each user input, Godot passes `gdctx` (regionId, tileId, nodePath, signals) into `H.Run.nextturn`, which then calls `H.Selector.choosePattern` with the current region/tile, user intent labels, routerState (tier), budgetState, and metricsState.[55]

Within H.Selector, the **history-selector rules** handle timing and pacing via metrics and history rather than raw clocks: SEL-R002 implements per-type cooldowns (for example, jumpscare 10 minutes, lore reveal 15 minutes), SEL-R004 implements an escalation curve based on session depth and budget ratio, and SEL-R005 enforces budget fit. So in practice, encounters trigger when the player speaks/acts, but whether that turn becomes a "heavy" encounter versus a mild beat depends on a combination of location (region/tile), cooldown history, budget, metric bands (DET, STCI, CDL), and selector patterns (ESCALATIONARC, REGIONALNARRATIVE, COOLDOWNFILLER) that gate which candidates are eligible.[55]

82. How H.Selector receives input (polling vs pub/sub)

In the current H.Run design, H.Selector is a **pure function** called synchronously from H.Run; it is not an autonomous pub/sub consumer. The canonical path is: on each turn, `H.Run.nextturn` classifies user input via H.Banter, updates banter state, samples invariants (CIC, SHCI, etc.) via H.Invariants, then calls:[55]

```
local selectorresult = H.Selector.choosePattern(
    gdctx.regionId or state.runContract.regionRef,
    gdctx.tileId,
    userIntent = labels,
    sessionId = sessionid,
    routerState = state.routerState,
    budgetState = budgetstate,
    metricsState = state.metricsState
)
```

This means H.Selector is effectively **polled per turn** by H.Run/H.Director, with all necessary context passed in (session, router, budget, metrics), and it in turn reads its own internal session/cross-session state (cooldowns, active patterns, encounter history) from the data described in `history-selector-rule-v1.json`. You could later back this with a pub/sub stream of events if you want, but the spec as written assumes "call on demand from H.Run," not an event bus.[55]

83. Exact payload from H.Director / H.Run to H.Selector

There are two closely related surfaces:

1. The **engine-side** state machine (`luahpcrunstatemachine.lua`) uses `H.Selector.choosepattern(regionRef, seedRef, sessionId, userIntent, tier, invariantsSnapshot)`.[55]
2. The **Godot bridge** (`luahrungodotbridge.lua`) uses `H.Selector.choosePattern(regionId, tileId, userIntent, sessionId, routerState, budgetState, metricsState)`.[55]

Putting those together, the effective payload is:

- Spatial context: `regionId` (or `regionRef`), `tileId` (or `seedRef` for seed-level patterns).[55]
- Session identity: `sessionId`.[55]
- Player intent: `userIntent` derived from H.Banter labels (intent tags, conflict flags).[55]
- Routing / entitlement: `routerState` (effective tier, permitted repos/contracts).[55]
- Safety & cost: `budgetState` (current horror intensity budget and per-modality budgets).[55]
- Entertainment metrics: `metricsState` or a snapshot of UEC, EMD, STCI, CDL, ARR.[55]
- Invariant snapshot (in the lower-level loop): CIC, SHCI, DET and other invariants for the current region/seed.[55]

H.Selector then merges this with its internal **sessionStateContract** and **crossSessionStateContract** (served IDs, active patterns, cooldown registry, last intensity, session depth, cross-session encounter history, lore thread progress) as specified in the history-selector rule schema.[55]

84. Fallback when no nodes meet metrics and invariants

The history-selector rules explicitly anticipate **exclusion** states where candidates are removed by SAF, consent, or budget rules. SEL-R001 delegates to H.Safety for a safety prefilter; SEL-R002, SEL-R005, and budget-fit logic can also exclude patterns when cooldowns or budget are violated; exclusions are final (no later rule can resurrect an excluded candidate).[55]

If, after applying all rules, no candidate remains or the chosen candidate's `resolutionStrategy` is `deny`, H.Run handles this via `H.Run.rendersoftrefusal`. This uses a special "conflict" template: it calls H.Templates.select with `intent = { conflict = true }`, `explicitnessBand = implied`, and `forceImplied = true`, to generate a diegetic **soft refusal** such as "this place cannot host that event; something holds the horror back," without exposing raw content. Designers can also ensure there is always at least one **COOLDOWNFILLER** pattern like `PAT-COOL-001 Safe Harbor` whose invariant and metric requirements are extremely permissive, so in many cases the fallback is a safe, low-DET beat instead of a refusal—this is why the pacing library includes Safe Harbor with minimal budget range and child-tier consent.[55]

85. Available selector_patterns (selection algorithms)

The selector pattern schema (`history-selector-pattern-v1.json`) and its initial library define **semantic pattern types** that correspond to different selection behaviors, not just raw "random vs greedy" flags. The initial catalog includes at least:[55]

- `ESCALATIONARC` patterns (for example, `PAT-ESC-001 Classic Slow Burn`, `PAT-ESC-002 Double Peak`) that define multi-beat arcs and enforce that exactly one escalation arc is active at all times; these behave like guided, stage-aware paths where SEL-R004's escalation curve rule heavily influences selection.[55]
- `REGIONALNARRATIVE` patterns (for example, `PAT-REG-001 Abandoned Ward Sequence`, `PAT-REG-002 Subterranean Descent`) that auto-activate based on entering certain PCG regions; these act like region-locked scripts with their own internal beat sequences.[55]
- `LORETHREAD` patterns (for example, `PAT-LORE-001 Foundation Mythos Thread`) that persist across sessions and unlock deeper lore when completed; they bias selection toward continuing the same thread until completion.[55]
- `COOLDOWNFILLER` patterns (for example, `PAT-COOL-001 Safe Harbor`) designed as low-cost, low-DET events used by cooldown rules and budget fallbacks.[55]
- `THEMATICCLUSTER` patterns (for example, `PAT-THM-001 Water and Submersion`) that group events by motif and are favored by diversity and regional affinity rules when under-represented.[55]

Algorithmically, the **history-selector rule pipeline** gives you:

- Random within the **remaining** candidate set, but **weighted** by a scoring function that starts at 1.0 and is modified by repetition penalties, escalation curves, budget fit, pattern match quality, diversity bonuses, and regional affinity; scores are clamped between 0.0 and 5.0.[55]
- Deterministic backbone behavior via pattern type rules: "exactly one ESCALATIONARC must be active," automatic activation of REGIONALNARRATIVE on region entry, cross-session persistence for LORETHREAD; these constraints effectively implement "predefined path" behaviors on top of the weighted selector.[55]

So instead of manually choosing "Random vs Greedy vs Pathfinding" at runtime, designers express intent by picking pattern types and rule parameters: ESCALATIONARC + strong SEL-R004 weights acts like a greedy escalation path; REGIONALNARRATIVE + region triggers acts like a path-following mode; diversity and regional affinity rules act like "soft pathfinding" that nudges the system toward under-explored motifs or spaces, all within the same scoring engine.[55]

⁂

| 86 | `[RUNTIME]` How is a 'node' defined in the context of the `H.Selector`? Is it a small snippet of dialogue, a full encounter, a map area, or a collection of entities? |

87	`[RUNTIME]` How do selected 'nodes' chain together to form a coherent narrative or sequence of events? Is there a graph-based structure that defines valid transitions?
88	`[RUNTIME]` How does the `H.Director` signal a shift in tone or pacing to the `H.Selector`? For instance, how would it transition from a 'low-tension' to a 'high-tension' selection mode?
89	`[RUNTIME]` What is the process for the `H.Director` to override the `H.Selector`'s choice if it violates an invariant or poses a safety risk?
90	`[RUNTIME]` How are 'liminal spaces' procedurally routed? Is the routing algorithm influenced by the `H.Director`'s goals or the player's actions?

In runtime terms, "nodes" are pattern-driven content decisions chosen by H.Selector on each turn, chained via session and cross-session state, and modulated by H.Director via metric and safety envelopes rather than ad-hoc overrides.[56]

86. What is a "node" for H.Selector?

In this stack a node is not just a dialogue line; it is a **content decision unit** defined by a history-selector pattern plus whatever template and VFX the engine binds to it. H.Selector works over a catalog of patterns described in `history-selector-pattern-v1.json` (ESCALATIONARC, REGIONALNARRATIVE, LORETHREAD, COOLDOWNFILLER, THEMATICCLUSTER), each with IDs like `PAT-ESC-001` and metadata about beats, content types, and budget ranges.[56]

At runtime, when `H.Run.nextturn` calls `H.Selector.choosePattern(regionId, tileId, userIntent, sessionId, routerState, budgetState, metricsState)`, the **selector result** (patternId, explicitnessBand, allowedEventTypes, etc.) effectively is the "node" chosen for that turn. Godot then uses this to pick a concrete AI-chat template (via H.Templates), drive map/VFX, or spawn entities tied to the pattern's type and region, so a node can manifest as a short beat, a multi-step encounter entry, or a spatial transition, but the selection logic always sees it as "one pattern choice this turn."[56]

87. How nodes chain into a coherent narrative

Coherence comes from the **graph encoded in selector patterns and session state**, not from H.Selector inventing structure on the fly. The pattern library already defines types with implicit graph edges:[56]

- ESCALATIONARC patterns define multi-beat arcs and a backbone rule: exactly one escalation arc is active at all times; entering or completing stages in that arc functions like traversing a path.[56]
- REGIONALNARRATIVE patterns activate automatically when entering a matching region and pause when leaving, so moving through the map replays a consistent sequence of nodes tied to that region.[56]
- LORETHREAD patterns track progress across sessions and only unlock deeper branches once earlier beats are served, giving you persistent narrative chains.[56]

The **history-selector rule** schema defines a `sessionStateContract` with fields like `servedContentIds`, `activePatterns`, `lastIntensity`, and `sessionDepth`, plus a `crossSessionStateContract` with encounter history and lore thread progress. Rules like repetition penalty (SEL-R003), pattern match scoring (SEL-R006), and diversity bonus (SEL-R007) act over these states, effectively treating them as a graph of what has been seen and what is eligible next. So yes, under the hood this is graph-like, but expressed as pattern types + rules over session/cross-session state, rather than a hardcoded graph editor in the runtime.[56]

88. How H.Director signals tone/pacing shifts to H.Selector

H.Director does not call H.Selector with a "mode = high tension" flag; it **shifts the envelopes** that H.Selector already respects. At session start, `H.Run.initsession` reads the runContract's `styleEnvelope.metricBands` (UEC, EMD, STCI, CDL, ARR) and `metricsEnvelope.invariantTargets` (DET caps, etc.) and initializes H.Metrics, H.Budget, H.Banter, H.Safety, and H.Selector with that configuration.[56]

On each turn, H.Run feeds current `budgetState` and `metricsState` into `H.Selector.choosePattern`, and the selector rules apply escalationcurve (SEL-R004) and budgetfit (SEL-R005) to favor patterns whose expected deltas move the run toward the target bands. A shift from low to high tension therefore happens when H.Director (via the personaContract + experienceContract) chooses to:[56]

- Allow higher DET/STCI bands for the current stage of the run (for example, moving from OuterThreshold to Rupture), and
- Let H.Budget/H.Metrics accumulate enough "room" that SEL-R004 and budget rules now favor higher-intensity patterns.[56]

In other words, H.Director **signals** pacing by picking experienceContracts and directorPersona profiles with particular curves and by letting H.Budget/H.Safety loosen or tighten clamps; H.Selector reads these bands every turn and naturally shifts from low-tension COOLDOWNFILLER to higher-tension ESCALATIONARC beats when the envelopes and rules say it is safe and on-target.[56]

89. Overriding H.Selector when invariants or safety are violated

Directorial "override" is handled as **governed denial and soft refusal**, not arbitrary replacement. Safety invariants are evaluated at multiple points:[56]

- H.Safety's chat-safety invariants (ABSOLUTE, CONSENT-GATED, BUDGET-GATED) provide hard caps and short-circuit rules.[56]
- The history-selector rules start with SEL-R001, which delegates to H.Safety as a prefilter; any candidate violating safety is **excluded** before scoring.[56]

If, despite this, a chosen pattern's `resolutionStrategy` is `deny` (for example, no candidate survives safety/budget/invariant checks), `H.Run.nextturn` does not let that choice through. Instead it calls `H.Run.rendersoftrefusal`, which:[56]

- Calls H.Templates.select with `intent = { conflict = true }`, `explicitnessBand = implied`, and `forceImplied = true`.
- Produces a narrative fallback like "this place cannot host that event; here is an implied alternative," which is then passed through boundary/redaction and logged.[56]

Additionally, the director persona safety adapter spec (`director-persona-safety-adapter.md`) introduces `H.Director.applySafetyDecision`, so persona logic always routes allow/soften/imply/refuse decisions through a single adapter, rather than trying to fight H.Safety or H.Selector. So the override process is: SAF or selector conflict → resolutionStrategy deny → H.Run soft-refusal or safe filler pattern → metrics and Dead-Ledger telemetry log the downshift.[56]

90. Procedural routing of liminal spaces

Liminal routing is handled by treating **liminal spaces as high-LSG / high-HVF regions** in the invariant field and letting selector and PCG logic respect those gradients. The unified Lua API lists LSG (Liminal Stress Gradient) and HVF (Haunt Vector Field) as invariants H.Invariants exposes for each region/tile; H.Selector's invariants snapshot includes these alongside CIC, SHCI, DET, etc.[56]

Procedural routing can then work in two coupled ways:

1. **H.Selector side:** patterns tagged as liminal or maze-like require certain LSG/HVF bands; SEL-R008 (regional affinity) and diversity rules will favor these patterns at liminal edges, causing more disorienting events and narrative beats when the player moves along high-LSG ridges or crosses thresholds.[56]
2. **Map/engine side:** the PCG layer treats the map as a graph whose edges are weighted by LSG/HVF and player History; routes through high-LSG areas can probabilistically loop, shift, or re-label nodes (for example, doors that sometimes send you back, hallways that visually repeat), while H.Director uses runContract style envelopes to cap how extreme this can get so CDL and DET stay within bands.[56]

Player actions influence this via History and metrics: choosing risky paths, ignoring stabilizing cues, or pushing high-DET choices can raise local LSG/HVF within the seed's envelope; selectors and PCG then respond by increasing the likelihood of liminal patterns and non-intuitive routes. H.Director's goals (for example, a "labyrinthine" experienceContract) determine how much of the run's distance should traverse high-LSG routes vs safer, low-LSG connectors, giving you disorientation that is systemic and tunable rather than random.[56]

⁂

| 91 | `[RUNTIME]` How do 'spectral entities' function as 'history queries'? What specific information from the `History` is encoded into their appearance, behavior, or dialogue? |

92	`[RUNTIME]` How is the `SHCI` updated when a spectral entity interacts with the player or the environment? What are the rules governing this corruption?
93	`[RUNTIME]` How does the `H.Director` decide when to introduce a 'liminal routing' event versus a standard encounter?
94	`[RUNTIME]` What is the in-world signaling mechanism for a change in the `DirectorPersona`? Is it a voiceover, a change in music, or subtle environmental shifts?
95	`[RUNTIME]` How are `BCI` (Blackout Control Interface) and `DeadLantern` responses rendered in the game world? Are they persistent UI elements or ephemeral visual/audio effects?

Spectral entities, SHCI drift, liminal routing, director shifts, and BCI/DeadLantern all ride on the same runtime surface: the unified H. Lua API, History state, and Dead-Ledger decisions, rather than ad-hoc "spooky" rules.[57]

91. Spectral entities as history queries

Spectral entities should be implemented as **visualized queries against History and invariants**, not freeform monsters. At the API level, H.Invariants exposes CIC, MDI, AOS, RRM, FCF, SPR, SHCI, HVF, LSG, etc. per region/tile, and H.Selector's state includes `encounterHistory`, `loreThreadProgress`, and `lifetimeTypeCounts` in the cross-session contract. A spectral archetype (defined in an Atrocity-Seed or spectral-seed registry entry) specifies which parts of History it binds to: for example, "echo of vanished workers" might pull prior encounters tagged with `accident-industrial`, the local CIC band, and any Dead-Ledger bundle attestation for that region.[57]

At runtime, spawning that entity means: query History for matching events, sample invariants via H.Invariants, then feed those into the entity's behavior template—appearance motifs driven by CIC/MDI/AOS, movement patterns influenced by HVF/LSG, and dialogue or implied "body language" that references already-served content IDs, unfinished lore threads, or prior failure states. Because H.Selector and Policy.DeadLedger only expose opaque IDs and invariant snapshots at Tier-1, the entity's dialogue never leaks raw vault content; it talks in abstractions ("you walked this corridor before and turned away") that are literally derived from `encounterHistory` and SHCI, giving you a **high SHCI** spectral-history coupling index in practice.[57]

92. Updating SHCI when entities interact (corruption rules)

SHCI in the unified API is part of the invariants used by H.Selector and H.ImageHelper and bound to runContract `metricsEnvelope.invariantTargets`; it should evolve only through well-defined rules, not ad-hoc increments. A sensible pattern is:[57]

- Treat each spectral entity interaction as a **corruption event** with a small, bounded SHCI delta defined in the spectral seed or Atrocity-Seed metadata (for example, +0.02 if the player accepts a bargain, –0.01 if they perform a cleansing action), and clamp SHCI to the seed's invariant envelope.
- Log each interaction via H.Metrics/H.Selector decision events, so Telemetry and Dead-Ledger can audit how SHCI changed.[57]

Implementation-wise, you can expose a helper in the invariants/selector layer (for example, `H.Invariants.applySpectralInteraction(sessionId, regionId, seedId, interactionKind)`) that reads the seed's allowed SHCI band, applies the configured delta for that `interactionKind`, and persists the new SHCI snapshot back into region/seed state for this session (and optionally into cross-session History for long-term corruption). Rules for corruption must respect consent and safety: SAF invariants and metric guardrails cap DET and CDL, so even if SHCI rises, selectors and H.Director are still constrained to safe bands per tier; as SHCI climbs, more history-coupled patterns and liminal routes become eligible, but they still pass through H.Safety and H.Budget gates.[57]

93. When H.Director chooses liminal routing vs standard encounters

H.Director's decisions are always expressed through **envelopes and constraints** that H.Selector and PCG respect, not direct imperative calls. The directorPersona and experienceContract define target bands for DET, STCI, CDL, ARR and also implicitly how much liminality (via LSG/HVF) the session is allowed to lean into. On each turn, H.Run consults H.Metrics and H.Budget, then calls H.Selector with the current invariants snapshot, budget, and metrics; selector rules can look at LSG/HVF to decide whether liminal patterns are eligible.[57]

In practice, H.Director "chooses" liminal routing by:

- Selecting an experienceContract / directorPersona whose style envelope calls for higher liminal exposure (for example, more time in high-LSG tiles or specific liminal pattern types).[57]
- Letting H.Budget and H.Safety decide whether high-tension liminal events are currently allowed; if not, standard encounters (REGIONALNARRATIVE, COOLDOWNFILLER) win.[57]

You can also add a simple policy field in director comfort policy (for example, `maxLiminalFraction` or `preferredLiminalStageRanges`) that H.Director uses to bias the pattern types passed into H.Selector: early stages might prefer standard encounters, mid-run allows liminal routing events, and late-run may require them when safe. Player actions influence this indirectly via metrics: repeatedly choosing risky or curiosity-driven options can push EMD/CIC/SHCI in ways that make liminal routes more attractive in the scoring function, but H.Director's policy still caps how often those can occur per run.[57]

94. In-world signaling for DirectorPersona changes

DirectorPersona is a Phase-2 contract specifying `directorKind`, tone, invariant bands, metric targets, and a `comfortPolicyRef`; changing it at runtime is a governance event, so the in-world signaling must respect both safety and immersion. The director Lua API spec proposes `H.Director.loadPersona(sessionContext, directorPersonaId)` and `H.Director.constrainMetrics(...)`; when a new persona is bound, the engine can use this as a cue to emit **diegetic signals** rather than UI pop-ups.[57]

Most coherent:

- **Voice layer**: shift narrator voice characteristics according to `toneLabel` (clinical vs ritual vs archivist) and allow a brief in-character line that acknowledges the change ("A different hand turns the pages now"), without

naming policies.

- **Audio / music**: adjust music mix and ambient SFX in tandem with the persona's DET/STCI bands—a more severe persona tightens soundscapes and dynamic range, while a gentler persona adds calmer textures.
- **Environment**: subtle palette, VFX, or UI frame changes keyed to the directorKind, making it readable but not breaking immersion (for example, archivist overlays, ritual chalk lines, cold overseer static).

Because SAF-008 (Director Persona Leash) is an ABSOLUTE invariant, any persona sharpening must stay inside tier caps; signals should therefore be toned as "new director flavor" rather than "now it's unsafe," reinforcing the notion that even a harsher persona is still governed.[57]

95. Rendering BCI and DeadLantern responses

BCI (Blackout Control Interface) and DeadLantern live in the **Dead-Ledger and entitlement layer**; the unified API describes Policy.DeadLedger.requestSpectralSeed, which takes a `bciSnapshot` and returns `decision`, `safetyTier`, `intensityBand`, and a `bciseedProfile`. In-world, these should manifest as **ephemeral controls and omens**, not cluttered HUDs.[57]

A clean split:

- **BCI**: primarily a hidden control channel; its decisions (for example, "allow-with-caps", "throttle intensity", "deny seed") are reflected through short-lived visual/audio cues and adjustments to encounters, not explicit UI. When BCI throttles or halts, you might briefly dim lights, glitch the environment, or mute entities, signaling that something in the "infrastructure" intervened.
- **DeadLantern**: a lore- and safety-facing artifact; Dead-Ledger proofs (bundle and spectral seed attestations) are never shown directly, but you can embody them as in-world lanterns or devices that flicker, darken, or flare when an underground artifact or spectral seed is gated or allowed.[57]

Neither should be persistent system UI in the main experience: to stay within the governed horror fantasy, treat them as **ephemeral effects**—brief overlays, sound stingers, diegetic devices that respond when Dead-Ledger decisions change what the player can see. Runtime still logs everything via NDJSON telemetry (including BCI decisions and any Dead-Ledger interactions), but players experience only atmospheric shifts, not protocol details.[57]

⁂

| 96 | `[RUNTIME]` What specific changes in the `metric_bands` or `invariant_state` trigger a visible change in the `BCI` or `DeadLantern`? |

97	`[RUNTIME]` How does the system respond if the player attempts to interact with the `BCI` or `DeadLantern`? Is this interaction allowed, discouraged, or forbidden?
98	`[RUNTIME]` What is the behavioral difference between a `DeadLantern` when the `CDL` is low versus when it is high?
99	`[RUNTIME]` How does the `H.Safety` module use the `AOS` invariant to prevent an encounter from becoming too intense? What is the intervention point?
100	`[RUNTIME]` How is the player's consent status (`CIC`) visually or audibly communicated to them during a session without explicitly stating it?

BCI and DeadLantern should behave like diegetic safety instruments: they react to numeric shifts in invariants/metrics and Dead-Ledger decisions, but players only ever see atmospheric changes, not raw numbers.[58]

96. Which metric/invariant changes visibly affect BCI / DeadLantern

BCI decisions come from `Policy.DeadLedger.requestSpectralSeed(seedDeadLedgerRef, bciSnapshot, context)`, which returns a `decision` (allow, deny, allow-with-caps), `safetyTier`, and an `intensityBand`. In practice you want visual changes when:[58]

- DET or explicitness budget breaches certain bands and the **intensityBand** in the decision drops (for example, from 7 to 5), meaning BCI is throttling seed intensity.
- Consent guardrails or metrics-ethics guardrails fire, causing `allow-with-caps` instead of `allow` for a spectral seed or bundle.[58]

DeadLantern should react when **session-level bands and invariant caps** are hit:

- When DET and CDL approach their caps from consent + director comfort policy (for example, near the "near-soft-cap" band in budget), DeadLantern dims or flickers to signal the system is about to downshift.[58]
- When SHCI or CIC cross thresholds that unlock or block spectral seeds (and a new Dead-Ledger decision token is issued or revoked), DeadLantern can flare briefly or change color to mark a deeper or sealed layer of history, aligned with the new `bciseedProfile` or bundle decision.[58]

So the triggers are not arbitrary; they map directly to: "BCI changed decision or intensityBand," "budget/DET hit near-soft-cap or hard caps," or "invariant caps from consent/director/region were reached," all of which H.Budget, H.Consent, and Policy.DeadLedger expose in structured results.[58]

97. Player interaction with BCI / DeadLantern

BCI is meant as a **control plane** between engine and Dead-Ledger, not as an in-world toy, so direct interaction should be minimal or tightly constrained. The safest pattern is:[58]

- Direct interaction with BCI itself (as a protocol object) is **forbidden** in-world; players can only interact with manifestations—like terminals or consoles—that the narrative pretends are "power" or "ritual" controls, while under the hood the engine may choose to call Policy.DeadLedger with a different context or refuse outright.[58]
- DeadLantern is a lore/safety artifact; interaction can be allowed but heavily rule-bound. Touching or focusing on it might request more information (`H.Consent.currentState`, comfort controls) or voluntarily lower intensity (for example, player chooses to "dampen" the lantern, mapping to a softer experienceContract), but it should never force the system to violate caps.[58]

When a player "tries" to push intensity up through these objects, H.Consent and H.Safety still have precedence: guardrails and invariants decide, then the interaction is narrated as "the lantern refuses" or "the blackout system locks the controls," preserving both safety and immersion.[58]

98. DeadLantern behavior at low vs high CDL

CDL is the cognitive dissonance load; low CDL means the experience is coherent and explanations are converging, high CDL means multiple conflicting readings are active. DeadLantern should mirror this:[58]

- **Low CDL:** the lantern behaves relatively **stable and legible**—steady glow, clear silhouettes, consistent audio motifs. It can hint at routes or past failures in a way that feels like guidance, reinforcing that the player's understanding is tracking the world.
- **High CDL:** the lantern becomes **erratic and ambiguous**—flickers, jumpy shadows, audio desync, color shifts that do not map cleanly to any single meaning. It might split into multiple overlapping afterimages or emit contradictory whispers, visually encoding that the narrative state is overloaded.

Because CDL is bounded by consent and director comfort policies, DeadLantern's most chaotic behaviors only appear inside allowed spans; the system never lets CDL climb into "pure noise" in public tiers, so the lantern's confusion signals dissonance without becoming incomprehensible or distressing beyond the configured caps.[58]

99. How H.Safety uses AOS to prevent over-intensity

AOS (Archival Opacity Score) measures how "thick" or occluded the historical record is; high AOS implies more missing, redacted, or contradictory data. In the safety and boundary layer, high AOS should act as a **soft brake** on explicitness and intensity:[58]

- H.Safety's invariants mapping ties each safety invariant to specific metrics and invariants; in high-AOS regions, rules can require higher ARR and lower CDL ceilings for certain content types, and they can project a tighter DET cap than the global consent cap.[58]
- At runtime, `H.Safety.evaluateTurn(sessionId, candidateOutput, context)` sees `regionId` and, via invariantCaps, the local AOS; if a candidate output would push DET, CDL, or explicitness beyond what high-AOS allows, it returns a decision of `soften` or `imply` with an appropriate redaction profile before the player ever sees it.[58]

The **intervention point** is therefore at output-gate: after candidate text is produced and budget has been updated, but before it is shown. If an event is too explicit for a high-AOS region, H.Safety softens or implies, and in severe cases (violating ABSOLUTE invariants or metrics-ethics guardrails) can force a refusal and prompt H.Run to select a softer fallback pattern instead.[58]

100. Conveying consent status (CIC) without explicit UI

CIC in this context is the Catastrophic Imprint Coefficient, while **consent state** is managed by H.Consent (`consentTier`, `invariantCaps`, explicitnessBudget). You do not want to tell players "your consent tier is adult-basic" mid-scene, but you do want them to feel how **much the world will let them push.**[58]

Good patterns:

- **Tone and available choices:** in lower consent tiers or when explicitnessBudget is low, high-DET/high-risk options are either absent or phrased more cautiously; the Director persona may warn more often ("this path is sealed to you") and emphasize safe exploration over transgressive acts. When caps are higher, more daring options appear and the Director's language becomes bolder—still within tier limits.
- **Environmental affordances:** doors, sigils, or rituals that correspond to higher DET/SHCI routes might appear visually present but inert (dim, unresponsive) at low consent, and become responsive or more vivid once the user opts in to a stronger experience (i.e., consent tier change), never exposing why, just that "something has allowed this now."
- **Subtle feedback:** when H.Consent.guardrails or budget guardrails trigger downshifts, reflect that as in-world events ("the house itself intervenes; the noise recedes") rather than a warning dialog, letting players infer that the system will protect them if they push too far.[58]

All of this uses the same caps that H.Consent and H.Budget compute—max DET, invariantCaps (CIC, SHCI, etc.), explicitnessBudget bands—but expresses them through narrative, choice design, and environment, so consent status is *felt* instead of spelled out.[58]

⁂

1. horrorplace-100-session-and-ru-CsI0CSAkRdmHsA34MNtuFA.md
2. 1-research-questions-and-items-iqkrJm1mT.OrdJYzXGfBjA.md
3. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
4. this-research-focuses-on-stren-hZWp2wBpQkW2xDO2IMK_LA.md
5. this-research-focuses-on-creat-b.QYu2gNRjaV4nBZ03r88w.md
6. this-research-focuses-on-compl-E8Y0Lhm_STKi3dQhIJsfSA.md
7. this-research-focuses-on-impro-YmKWzHr3TC6EZLDY.tmO1w.md
8. 1-should-the-research-prioriti-tQnn6sdDQ06XDNmNoVKx.g.md
9. this-research-focuses-on-advan-Gn_aMO_jQvqfg7wXixdLTw.md
10. suggest-the-next-task-below-in-BvOUg9mxQKKnfOeCjfxKsg.md
11. CHAT_DIRECTOR v1 Implementation Research — Module Specifications.docx
12. break-down-the-directory-in-up-hsLbuD2NRu29.S35fpB6Yg.md
13. the-algorithmic-architecture-o-A7N0Cgi2QFqkAHFdhxjM2g.md
14. this-research-focuses-on-desig-DemATE1ZRtOBxLRQlhB93g.md
15. from-prompt-to-praxis-a-contra-7os9eunGTtCvPKl3BSsZXg.md
16. this-research-focuses-on-impro-RixUoveJRciBLt4RY0p1Vw.md
17. which-of-these-memory-architec-dnlg0pjJTqu4DFJ3.oFZgw.md
18. this-research-focuses-on-devel-Eh3ThsQBQg.r6HDAZZhx5w.md
19. formal-definition-and-integrat-V8UK1uYHRKez9vQ0Xfga2A.md
20. this-research-focuses-on-enhan-2EXdmBbbTneisQNtSJfTWA.md
21. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
22. formal-definition-and-integrat-V8UK1uYHRKez9vQ0Xfga2A.md
23. this-research-focuses-on-advan-Gn_aMO_jQvqfg7wXixdLTw.md
24. this-research-focuses-on-creat-b.QYu2gNRjaV4nBZ03r88w.md
25. 1-research-questions-and-items-iqkrJm1mT.OrdJYzXGfBjA.md
26. horrorplace-100-session-and-ru-CsI0CSAkRdmHsA34MNtuFA.md
27. this-research-focuses-on-compl-E8Y0Lhm_STKi3dQhIJsfSA.md
28. Horror$Place VM-Constellation Deep Audit Report.pdf
29. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md

30. this-research-focuses-on-advan-Gn_aMO_jQvqfg7wXixdLTw.md
31. horrorplace-100-session-and-ru-CsI0CSAkRdmHsA34MNtuFA.md
32. formal-definition-and-integrat-V8UK1uYHRKez9vQ0Xfga2A.md
33. horrorplace-100-session-and-ru-CsI0CSAkRdmHsA34MNtuFA.md
34. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
35. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
36. horrorplace-100-session-and-ru-CsI0CSAkRdmHsA34MNtuFA.md
37. horrorplace-100-session-and-ru-CsI0CSAkRdmHsA34MNtuFA.md
38. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
39. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
40. horrorplace-100-session-and-ru-CsI0CSAkRdmHsA34MNtuFA.md
41. horrorplace-100-session-and-ru-CsI0CSAkRdmHsA34MNtuFA.md
42. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
43. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
44. horrorplace-100-session-and-ru-CsI0CSAkRdmHsA34MNtuFA.md
45. horrorplace-100-session-and-ru-CsI0CSAkRdmHsA34MNtuFA.md
46. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
47. horrorplace-100-session-and-ru-CsI0CSAkRdmHsA34MNtuFA.md
48. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
49. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
50. horrorplace-100-session-and-ru-CsI0CSAkRdmHsA34MNtuFA.md
51. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
52. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
53. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
54. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
55. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
56. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
57. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
58. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md